МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ПРИКЛАДНОЇ МАТЕМАТИКИ ТА ІНФОРМАТИКИ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Допускається до захисту

Завідувач кафедри

_____ _____

«_____» _____ 2025 р.

КУРСОВА РОБОТА

на тему:

# CAD-РЕДАКТОР ДЛЯ МОДЕЛЮВАННЯ ГЕКСАЕДРАЛЬНИХ СІТОК З АЛГОРИТМОМ ЛОКАЛЬНОГО ЗГУЩЕННЯ

Виконав:
студент групи ПМІм-12
напряму підготовки
«Інформаційні системи»
Мельник Остап Олегович

Науковий керівник:
доцент кафедри
інформаційних систем
Вовк Володимир Дмитрович

Львів — 2025

ЗМІСТ

ВСТУП . . . 3
РОЗДІЛ 1. ОГЛЯД ІСНУЮЧИХ РІШЕНЬ ТА АНАЛІЗ ПРОБЛЕМИ . . . 7
1.1 Огляд існуючих CAD/CAE систем і МСЕ-пакетів . . . 7
1.2 Методи генерації та уточнення сіток . . . 10
1.3 Порівняльний аналіз . . . 12
1.4 Виявлені недоліки існуючих рішень . . . 13
1.5 Висновки до розділу . . . 14
РОЗДІЛ 2. АЛГОРИТМ ЛОКАЛЬНОГО АДАПТИВНОГО УТОЧНЕННЯ . . . 15
2.1 Концепція двох просторів . . . 15
2.2 Загальна схема алгоритму уточнення . . . 16
2.3 Шаблони розбиття . . . 19
2.4 Маркування вузлів та парування шарів . . . 21
2.5 Три проходи: I → J → K . . . 22
2.6.6 Node Propagation — класифікація кейсів . . . 25
2.7 Анізотропне уточнення . . . 33
2.8 Транзакційний механізм сесії уточнення . . . 34
2.9 Складність алгоритму . . . 34
2.11 Висновки до розділу . . . 36
РОЗДІЛ 3. МЕТОДИ ПОКРАЩЕННЯ ЯКОСТІ СІТКИ ПІСЛЯ УТОЧНЕННЯ . . . 37
3.1 Що таке якість скінченно-елементної сітки . . . 37
3.2 Аналіз методів згладжування та вибір підходу . . . 38
3.3 Метрики якості та рекомендовані значення . . . 45
3.6 Зв'язок якості сітки з точністю . . . 45
3.7 Висновки до розділу . . . 46
ВИСНОВКИ . . . 46
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ . . . 49

# ВСТУП

Актуальність теми. Комп'ютерне моделювання фізичних процесів стало невід'ємною частиною сучасного інженерного проектування. Перш ніж виготовити деталь або конструкцію, її поведінку під навантаженням перевіряють числовими методами — передусім методом скінченних елементів (МСЕ). Це дозволяє ще на стадії проектування виявити небезпечні зони концентрації напружень, оцінити запас міцності та уникнути дорогих реальних випробувань.

Якість числового розв'язку в МСЕ безпосередньо залежить від того, наскільки добре побудована розрахункова сітка. Серед усіх типів скінченних елементів гексаедральні елементи (шестигранники, або «цеглинки») вважаються найточнішими: при однаковій кількості невідомих вони дають меншу похибку, ніж тетраедри. Саме тому в задачах механіки деформівного твердого тіла, геомеханіки та аеродинаміки гексаедральні сітки є бажаним, а часто й необхідним вибором.

Головна практична проблема полягає в тому, що зони підвищеного інтересу — наприклад, місце різкого звуження деталі, зварний шов або межа між шарами матеріалу — займають малу частину всього об'єму деталі. Якщо дрібнити сітку рівномірно по всьому об'єму, загальна кількість елементів зростає в $n_I \cdot n_J \cdot n_K$ разів, де $n_I, n_J, n_K \geq 2$ — кількість частин вздовж кожної з трьох осей. Це швидко вичерпує обчислювальні ресурси: сітка, що містить мільйон елементів, вимагатиме гігабайти оперативної пам'яті і годин процесорного часу.

Альтернативою є локальне адаптивне уточнення — коли дрібнення виконується лише там, де воно дійсно необхідне. Для тетраедральних сіток такий підхід відносно добре реалізовано в ряді пакетів. Для гексаедральних сіток ситуація значно гірша: при поділі одного гексаедра сусідні елементи мають бути відповідно адаптовані, щоб не утворилися так звані «висячі вузли» — вузли, що лежать на ребрі або грані сусіднього нерозбитого елемента. Висячі вузли порушують умову нерозривності поля переміщень і роблять результати МСЕ некоректними. Саме через цю складність більшість сучасних систем або не підтримують шаблонне локальне уточнення вже побудованої гексаедральної сітки без повторної генерації, або вирішують проблему висячих вузлів через глобальне перебудування сітки.

Ще одним важливим аспектом є доступність інструменту. Більшість потужних систем (ANSYS, Abaqus, COMSOL) є комерційними з високою вартістю ліцензій. ANSYS Mechanical коштує близько \$22 000 за постійну ліцензію або від \$10 000 на рік за підписку, а повний пакет сягає \$50 000+. Abaqus пропонує річну оренду від \$18 000 або постійну ліцензію від \$31 000–\$37 000 плюс \$8 500/рік на обслуговування. COMSOL стартує від \$3 500/рік за базовий модуль, проте кожен додатковий модуль коштує ще \$600–\$4 500, тому реальна вартість повноцінного пакету становить \$15 000–\$30 000 на рік. Відкриті альтернативи (Gmsh, FreeCAD) суттєво поступаються за функціональністю в частині гексаедральних сіток. Крім того, всі ці системи є desktop-застосунками: щоб скористатися ними, потрібно встановити важкий пакет програм

на конкретний комп'ютер. Веб-орієнтованого рішення, доступного з браузера без встановлення, серед вільно доступних інструментів наразі не існує.

Таким чином, задача розробки відкритої веб-орієнтованої системи для інтерактивного локального адаптивного уточнення гексаедральних сіток є практично актуальною та науково значимою.

Мета і завдання дослідження. Метою даної роботи є розробка та програмна реалізація інтерактивного веб-редактора гексаедральних скінченно-елементних сіток із підтримкою локального адаптивного уточнення.

Для досягнення поставленої мети вирішуються такі завдання:

1. Провести критичний аналіз існуючих систем генерації та уточнення скінченно-елементних сіток: визначити, яку функціональність вони надають, де є прогалини, і чому ці прогалини досі не заповнені.

2. Розробити ефективну топологічну структуру даних для гексаедральних сіток, яка підтримує швидкий обхід сусідів, ланцюжкову нумерацію та ієрархію розбиттів без прив'язки до координатної сітки.

3. Запропонувати та реалізувати алгоритм локального адаптивного уточнення на основі шаблонного підходу для двовимірних і тривимірних сіток, що гарантує конформність результуючої сітки.

4. Розробити метод поширення вузлів (Node Propagation) як механізм усунення каскадного застосування шаблонів при взаємодії між різними проходами алгоритму.

5. Реалізувати процедури покращення геометричної якості сітки після уточнення: лапласівське згладжування із вирівнюванням площ елементів та захистом від інверсії вузлів.

6. Розробити веб-орієнтовану архітектуру системи: серверна частина на .NET 8, клієнтська на Angular 17 із тривимірною візуалізацією через WebGL та Three.js.

Об'єкт дослідження — процес інтерактивного локального адаптивного уточнення гексаедральних скінченно-елементних сіток.

Предмет дослідження — алгоритми шаблонного уточнення гексаедральних сіток, метод поширення вузлів для забезпечення конформності, метрики якості сіток та методи їх покращення, архітектура веб-системи для інтерактивного редагування сіток.

Методи дослідження. В роботі використано методи топологічного аналізу (побудова ланцюжків, графи суміжності), методи оцінки якості сіток (масштабований якобіан та співвідношення сторін для 2D і 3D; мінімальний та максимальний внутрішній кут — додатково для 2D), методи геометричного згладжування вузлів. Програмна

реалізація побудована на принципах об'єктно-орієнтованого програмування та чистої архітектури з розділенням на шари.

Наукова новизна отриманих результатів. У результаті виконання роботи отримано такі нові результати.

1. Концепція двох просторів. Запропоновано представлення сітки у двох незалежних просторах: ідеальному (топологія з цілочисельними координатами) та фізичному (геометрія через ізопараметричне перетворення). Це дозволяє виконувати алгоритм уточнення незалежно від конкретної геометрії елементів і застосовувати один і той самий код до сіток будь-якої форми.

2. Метод поширення вузлів (Node Propagation). Запропоновано та реалізовано новий механізм, що уникає каскадного застосування базового шаблону T1 при взаємодії між різними проходами алгоритму. Ключова ідея: замість послідовного застосування двох T1 система відстежує грані з вузлами попереднього поділу і вибирає шаблон T2 або T4, що одночасно враховує розбиття по двох або трьох осях. Це запобігає погіршенню якості сітки — каскадне застосування T1 призводить до надто гострих кутів елементів і деградації форми.

3. Топологічне парування ланцюжків (Enumerate / BuildChains). Розроблено алгоритм глобальної нумерації ланцюжків елементів без прив'язки до координатної сітки. Алгоритм забезпечує детерміновану та відтворювану нумерацію незалежно від порядку маркування та конкретної конфігурації вже розбитих елементів.

4. Анізотропне узагальнення шаблонів. Шаблони T1, T2 та T4 узагальнено для довільних цілих параметрів поділу $n_I$, $n_J$, $n_K$ по кожній з трьох просторових осей. Це дозволяє виконувати анізотропне уточнення — наприклад, розбити елемент на $4 \times 1 \times 2$ частини — без зміни базової структури алгоритму.

5. Транзакційний механізм сесії уточнення. Розроблено протокол begin/cancel/commit для операцій уточнення, що дозволяє відкочувати незавершені або небажані зміни сітки без порушення її цілісності та без повторної генерації.

Практичне значення результатів. Результати роботи реалізовано у вигляді програмної системи CadEditor — інтерактивного веб-редактора гексаедральних сіток. Система дозволяє будувати сітки, локально їх уточнювати у довільних зонах, покращувати геометричну якість та експортувати результат у форматах Gmsh (.msh), VTK (.vtk) і Abaqus (.inp) для подальшого використання в розрахункових системах. Інтерфейс доступний у будь-якому браузері без встановлення додаткового програмного забезпечення. Система має відкрите REST API, що дозволяє інтегрувати її у ширші інженерні конвеєри. Реалізовані алгоритми придатні для застосування в задачах

механіки конструкцій, геомеханіки, біомеханіки та будь-яких інших областях, де потрібна якісна гексаедральна сітка з локальним уточненням.

Використання засобів штучного інтелекту. Під час виконання роботи для прискорення деяких рутинних задач розробки частково використовувались інструменти штучного інтелекту. Зокрема, мовні моделі застосовувались для генерації шаблонного коду (методи `ToString`, базові конструктори, налаштування залежностей IoC-контейнера), написання частини юніт-тестів для перевірки крайових випадків, а також для допомоги з оформленням формул у форматі LaTeX при підготовці цього звіту. Всі алгоритмічні та архітектурні рішення, а також ключові частини коду, розроблено самостійно.

Структура та обсяг роботи. Курсова робота складається зі вступу, трьох розділів основної частини, висновків та списку використаних джерел. Вступ описує актуальність теми, мету, завдання та наукову новизну. Перший розділ містить критичний огляд існуючих систем генерації та уточнення сіток з визначенням їх недоліків. Другий розділ детально описує розроблений алгоритм локального адаптивного уточнення гексаедральних сіток. Третій розділ розглядає методи покращення геометричної якості сітки після уточнення.

# РОЗДІЛ 1
# ОГЛЯД ІСНУЮЧИХ РІШЕНЬ ТА АНАЛІЗ ПРОБЛЕМИ

## 1.1 Огляд існуючих CAD/CAE систем і МСЕ-пакетів

Перед тим як описувати власну розробку, варто зрозуміти, які інструменти вже існують і чому вони не вирішують поставлену задачу. Нижче розглянуто вісім найпоширеніших систем — від безкоштовних відкритих до дорогих комерційних пакетів. Для кожної визначено, що вона вміє і де є обмеження стосовно локального уточнення гексаедральних сіток.

### 1.1.1 Gmsh

Gmsh — це вільно поширювана система генерації скінченно-елементних сіток з відкритим вихідним кодом. Вона широко використовується в академічному середовищі завдяки зручному скриптовому API та підтримці тривимірної геометрії через OpenCASCADE. Gmsh вміє будувати тетраедральні та змішані сітки через алгоритми Delaunay, Frontal та Netgen.

Що стосується гексаедральних сіток: Gmsh може генерувати структуровані гексаедральні сітки лише для геометрично простих тіл — паралелепіпедів або тіл, утворених переміщенням плоского перерізу вздовж прямої (труби, балки, призми). Для будь-якої іншої тривимірної геометрії Gmsh автоматично переходить до тетраедрів або гібридних сіток, де гексаедри є лише частиною.

Недоліки. Локального адаптивного уточнення вже побудованої гексаедральної сітки у Gmsh принципово не існує. Якщо після розрахунку виявилось, що в певній зоні потрібна вища точність, єдиний варіант — видалити всю сітку, змінити параметри розміру елементів і згенерувати її заново з нуля. Це означає, що весь обчислювальний час попереднього розрахунку витрачається даремно. Для великих сіток з мільйонами елементів повторна генерація може тривати десятки хвилин. Крім того, Gmsh — це desktop-застосунок, який треба встановлювати на кожну машину. Попри наявність C++/Python API, інтеграція у веб-застосунок вимагає складного серверного прошарку. Відсутня будь-яка підтримка транзакційних операцій — якщо генерація перервалась, результат втрачається повністю.

### 1.1.2 ANSYS Meshing

ANSYS Meshing входить до складу комерційного пакету ANSYS Workbench. Це потужна промислова система з підтримкою автоматичної генерації гексаедральних, тетраедральних та гібридних сіток. Алгоритм MultiZone автоматично ділить складну геометрію на прості прямокутні блоки і в кожному будує повністю структуровану гексаедральну сітку — тобто всі елементи вирівняні вздовж осей блоку. Алгоритм Sweep

призначений для тіл що мають однаковий поперечний переріз по всій довжині: він будує плоску сітку на одному торці і послідовно копіює її шар за шаром до іншого торця.

Недоліки. Попри потужність алгоритмів генерації, ANSYS Meshing не підтримує справжнього локального уточнення вже побудованої сітки. Те, що в ANSYS називається «local mesh refinement», насправді є лише заданням дрібнішого цільового розміру елементів у певній зоні з подальшою повною перегенерацією — тобто стара сітка знищується і будується нова. Окрім того, ANSYS Meshing є виключно комерційним продуктом з ціною ліцензії від \$10 000/рік, що унеможливлює його використання у навчальних або дослідницьких проектах без значного фінансування. Система не має жодного веб-інтерфейсу — лише Windows/Linux desktop-застосунок. Інтеграція у власні системи через API вимагає окремої ліцензії на скриптинг і наявності встановленого повного пакету ANSYS Workbench на тому ж комп'ютері.

### 1.1.3 Abaqus/CAE

Abaqus/CAE від компанії Dassault Systèmes — це одна з найпотужніших систем МСЕ-аналізу для задач механіки деформівного твердого тіла. Препроцесор має розвинені алгоритми побудови гексаедральних сіток (Medial Axis, Advancing Front), здатні обробляти складні промислові геометрії.

Недоліки. Навіть в Abaqus «локальне уточнення» насправді є маніпуляцією з Seeds — набором точок що задаються вручну на ребрах геометрії. Після зміни Seeds система повністю перебудовує сітку від початку. Жодної можливості взяти вже готову сітку і підвищити густину лише в одному місці без повної перегенерації не існує. Це критична проблема для ітеративних розрахунків: якщо результат МСЕ показав концентрацію напружень в певній зоні, інженер змушений вручну перезадати Seeds, дочекатись повної регенерації (яка може тривати години для складних геометрій) і запустити розрахунок заново. Вартість ліцензії Abaqus починається від \$18 000/рік за оренду або від \$31 000 за постійну ліцензію, що робить його недоступним для більшості університетських лабораторій. Системи не має веб-інтерфейсу, доступна тільки як важкий desktop-застосунок під Windows/Linux.

### 1.1.4 COMSOL Multiphysics

COMSOL — мультифізична система, відома зручним GUI та підтримкою широкого кола фізичних задач: механіка, теплообмін, електромагнетизм, акустика. Генерація сіток у COMSOL повністю автоматична: користувач задає лише глобальний або локальний розмір елементів, а система сама вирішує, яку топологію будувати. Гексаедральні елементи підтримуються тільки для структурованих сіток на прямокутних доменах — тобто коли кожна клітинка сітки відповідає чітко заданій позиції у регулярній сітці вузлів.

Недоліки. Будь-яка спроба «локального уточнення» в COMSOL закінчується тим

самим: система перегенеровує всю сітку з нуля з оновленими параметрами розміру. Шаблонного локального уточнення вже існуючої сітки немає в принципі. Для задач із складною геометрією COMSOL взагалі не будує чисто гексаедральні сітки — автоматика переходить до тетраедрів. Система є повністю закритою: вихідний код недоступний, внутрішній сітковий движок не можна інтегрувати у власні застосунки. Вартість ліцензії стартує від \$3 500/рік за базовий модуль, проте кожен додатковий фізичний модуль коштує ще \$600–\$4 500, тому реальна вартість повноцінного пакету становить \$15 000–\$30 000/рік. Веб-доступу немає.

### 1.1.5 Cubit / Coreform Cubit

Cubit (сьогодні — Coreform Cubit) є найбільш спеціалізованою системою для гексаедральних сіток. У ній реалізовано алгоритм Paving для квадрилатеральних сіток та його тривимірне розширення Plastering. Cubit підтримує «hex sweeping», «hex dominant» та блокові методи і здатний будувати якісні гексаедральні сітки навіть для складних промислових геометрій.

Недоліки. Попри те що Cubit є лідером у галузі hex meshing, навіть він не підтримує шаблонного локального уточнення вже побудованої сітки. Для отримання дрібнішої сітки в зоні інтересу Cubit пропонує єдиний варіант — повну перегенерацію з іншими параметрами. Cubit орієнтований на великі промислові та суперкомп'ютерні розрахунки та великі промислові проекти: інтерфейс складний, вимагає тижнів навчання, не адаптований для швидкого інтерактивного використання. Система є комерційною (безкоштовна версія Trelis має суттєво урізаний функціонал і не придатна для реальних задач), не має жодного веб-інтерфейсу та відкритого API.

### 1.1.6 Salome Platform

Salome — відкрита платформа для попередньої та подальшої обробки MCE-задач, що розвивається переважно у французьких наукових лабораторіях (CEA, EDF). Модуль SMESH підтримує тетраедральні сітки через Netgen та BLSURF, а також має обмежені можливості для гексаедральних сіток через блокове розбиття.

Недоліки. Локальне уточнення в Salome реалізовано виключно для тетраедральних сіток — гексаедральні сітки локально уточнювати неможливо взагалі. Інтерфейс системи застарілий і незручний: навіть прості операції вимагають багатокрокових діалогів. Документація неповна, значна частина функцій недокументована або задокументована тільки французькою. Система є виключно desktop-застосунком під Linux/Windows, без будь-якої веб-підтримки. Спільнота значно менша ніж у Gmsh, підтримка обмежена.

### 1.1.7 OpenFOAM (snappyHexMesh)

OpenFOAM — відкрита система для обчислювальної гідродинаміки (CFD), що має власний генератор сіток snappyHexMesh. Цей генератор будує гексаедрально-домінантні сітки на основі октарного дерева (octree) та підтримує локальне подрібнення через «refinement regions» — зони в яких рівень поділу октарного дерева збільшується.

Недоліки. Підхід snappyHexMesh принципово відрізняється від шаблонного уточнення структурованих гексаедральних сіток. По-перше, результуюча сітка не є структурованою: вона містить перехідні шари, зрізані клітинки та елементи неправильної форми біля меж геометрії. По-друге, такі сітки придатні для CFD-задач (течія рідини), але дають незадовільну точність для задач теорії пружності, де потрібні правильні гексаедральні елементи з хорошим якобіаном. По-третє, snappyHexMesh — це повністю пакетний інструмент командного рядка: жодного інтерактивного GUI не існує, налаштування виконуються через багаторядкові текстові конфігурації, налагодження — вручну через перегляд логів. Для інженера без досвіду CFD використання практично неможливе.

### 1.1.8 FreeCAD з модулем FEM

FreeCAD — відкрита параметрична CAD-система з модулем МСЕ (FEM Workbench), що використовує Gmsh як сітковий движок. Система безкоштовна і орієнтована на навчання та прототипування.

Недоліки. Оскільки FreeCAD повністю покладається на Gmsh для генерації сіток, він успадковує всі його обмеження: відсутність локального уточнення гексаедральних сіток, неможливість роботи з чисто гексаедральними сітками для складних геометрій, повна перегенерація при будь-яких змінах. FEM Workbench значно поступається спеціалізованим системам за функціональністю: відсутні розвинені контакти, обмежена підтримка нелінійних задач, немає постпроцесора для деформованої форми. Система нестабільна — часті краші при роботі з великими геометріями є відомою проблемою спільноти. Веб-доступ або API відсутні повністю.

## 1.2 Методи генерації та уточнення сіток

Щоб краще зрозуміти, чому локальне гексаедральне уточнення є складним завданням, розглянемо основні методи та їх властивості.

### 1.2.1 Методи генерації сіток

Методи генерації скінченно-елементних сіток поділяються на кілька класів.

Структуровані сітки будуються за регулярним шаблоном: вузли розміщуються в правильній сітці $(i, j, k)$, а кожен елемент визначається положенням у цій сітці. Такі

сітки прості у реалізації та дають якісні гексаедральні елементи, але застосовні тільки до простих доменів (паралелепіпеди, тіла обертання).

Неструктуровані сітки (Delaunay, Advancing Front) підходять для довільних геометрій. Для тривимірних об'єктів ці методи переважно генерують тетраедри. Побудувати неструктуровану суто гексаедральну сітку для довільного тривимірного тіла досі є відкритою науковою задачею.

Блокові (multi-block) методи є компромісом: геометрія вручну або автоматично розбивається на блоки, кожен із яких має просту форму, і для кожного блоку будується структурована сітка. Cubit та ANSYS MultiZone використовують саме цей підхід.

### 1.2.2 Стратегії уточнення сіток

Після початкової генерації сітки можуть виникнути зони, де потрібно вища точність розв'язку. Для цього використовуються три основні стратегії.

Глобальне рівномірне уточнення: всі елементи поділяються однаково. При кожному кроку загальна кількість елементів зростає у $n_I \cdot n_J \cdot n_K$ разів, де $n_I, n_J, n_K \geq 2$ — кількість частин вздовж кожної осі (наприклад, при $n_I = n_J = n_K = 2$ кількість елементів збільшується у 8 разів). Метод простий у реалізації, але вкрай неефективний: якщо зона інтересу займає 1% об'єму, 99% обчислювальних ресурсів витрачається на непотрібне подрібнення.

Адаптивне h-уточнення: система спочатку виконує МСЕ-розрахунок на грубій сітці, а потім аналізує результат — де напруження або переміщення змінюються надто різко між сусідніми елементами, там точність недостатня. Саме ці зони подрібнюються, після чого розрахунок повторюється. Для тетраедральних сіток цей підхід добре відпрацьований і широко застосовується. Для гексаедральних — значно складніший, оскільки подрібнення одного елемента породжує висячі вузли на сусідніх гранях, що руйнує конформність сітки.

Шаблонне локальне уточнення: для кожного елемента, що потребує уточнення, вибирається один з фіксованого набору шаблонів розбиття залежно від маркованих вузлів або граней. Метод гарантує конформність і передбачуване зростання кількості елементів. Саме цей підхід реалізовано у MeshBuilder.

### 1.2.3 Проблема висячих вузлів

При локальному уточненні гексаедральних сіток головна трудність — це уникнення висячих вузлів. Нехай елемент A розбивається на два дочірніх елементи A1 та A2. На спільному ребрі між A та його сусідом B з'являється новий вузол. Цей вузол «висить» на ребрі B: він існує з боку A, але немає з боку B.

Якщо залишити сітку в такому стані, функції форми елемента B не визначені в точці нового вузла, що означає розрив поля переміщень на грані AB. Це порушує основну умову МСЕ — нерозривність поля в межах сітки.

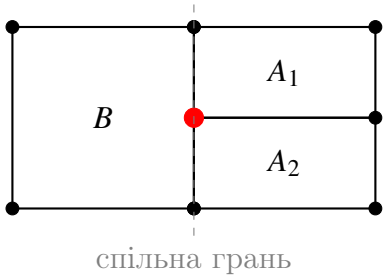



Рис. 1: Висячий вузол при локальному уточненні: елемент $A$ розбито на $A_1$ та $A_2$, на спільній грані з $B$ з'являється вузол (червоний), якого немає у $B$.

Сучасні комерційні системи (ANSYS, Abaqus) частково вирішують проблему через багатоточкові обмеження (MPC, Multi-Point Constraints). Ідея полягає в наступному: нехай вузол $P$ є висячим і лежить на ребрі елемента $B$ між його вузлами $B_1$ та $B_2$. Тоді переміщення $\mathbf{u}(P)$ виражається через переміщення $\mathbf{u}(B_1)$ та $\mathbf{u}(B_2)$ за допомогою лінійної інтерполяції:

$$\mathbf{u}(P) = (1-\xi)\,\mathbf{u}(B_1) + \xi\,\mathbf{u}(B_2), \tag{1}$$

де $\xi \in [0,1]$ — нормована координата положення $P$ на ребрі $B_1B_2$. Це обмеження додається до глобальної системи рівнянь МСЕ у вигляді додаткового рядка, що «прив'язує» ступені свободи вузла $P$ до вузлів $B_1$ і $B_2$, не розбиваючи елемент $B$. Для другого порядку (20-вузлові елементи) замість лінійної використовується квадратична інтерполяція з трьох вузлів.

Проте офіційна документація Abaqus прямо застерігає: використання MPC для локального уточнення може призводити до надмірного защемлення конструкції та похибок при розрахунку вигину — особливо для тонкостінних конструкцій. Шаблонне уточнення, реалізоване у MeshBuilder, уникає цих проблем повністю: сітка залишається конформною без жодних додаткових обмежень, а переміщення у спільних вузлах збігаються точно, а не наближено.

## 1.3 Порівняльний аналіз

У таблиці 1.1 зведено порівняння ключових характеристик розглянутих систем та розробленої системи MeshBuilder.

Як видно з таблиці, жодна з розглянутих систем не забезпечує одночасно трьох характеристик: шаблонне локальне уточнення гексаедральних сіток, веб-орієнтований інтерфейс та відкрите API для інтеграції. MeshBuilder заповнює цю прогалину.

Варто підкреслити відмінність між шаблонним уточненням (MeshBuilder) та octree-уточненням (snappyHexMesh): octree генерує сітки зі складними перехідними шарами і спрямоване на аерогідродинамічні задачі зі складною поверхнею, тоді як шаблонне уточнення зберігає регулярну гексаедральну структуру і підходить для задач

Табл. 1: Порівняння систем генерації та уточнення сіток

| Система | Ліцензія | Локальне hex уточнення | Веб-інтерфейс | Відкрите API |
|---|---|---|---|---|
| Gmsh | Відкрита | Ні | Ні | Часткове |
| ANSYS | Комерційна | Ні (глобальне) | Ні | Тільки з ліцензією |
| Abaqus | Комерційна | Ні | Ні | Ні |
| COMSOL | Комерційна | Ні | Ні | Ні |
| Cubit | Комерційна | Ні | Ні | Часткове |
| Salome | Відкрита | Ні (тільки тетр.) | Ні | Так |
| snappyHexMesh | Відкрита | Octree (не шаблонне) | Ні | Конфіг. файли |
| FreeCAD | Відкрита | Ні (через Gmsh) | Ні | Так (Python) |
| MeshBuilder | Відкрита | Так (шаблонне, T1/T2/T4) | Так | Так (REST + SignalR) |

механіки з рівномірними граничними умовами.

## 1.4 Виявлені недоліки існуючих рішень

На підставі проведеного огляду виділено п'ять ключових недоліків існуючих рішень.

1. Відсутність справжнього локального уточнення. Жодна з розглянутих систем не дозволяє уточнити вже побудовану гексаедральну сітку в довільній зоні без її повної перегенерації. Це означає, що кожна ітерація адаптивного циклу «розрахунок — оцінка похибки — уточнення» коштує стільки ж обчислень, скільки перша генерація сітки.

2. Проблема конформності. Ті системи, що підтримують локальне уточнення, зазвичай отримують сітки з висячими вузлами. Такі сітки придатні для деяких методів (наприклад, DG — метод Галеркіна з розривними функціями форми, або XFEM — розширений МСЕ що дозволяє тріщинам проходити крізь елементи без перебудови сітки), але несумісні з класичним МСЕ без додаткових процедур обробки.

3. Топологічна залежність від геометрії. Більшість алгоритмів уточнення орієнтовані на конкретну геометричну форму елементів (наприклад, «додай вузол в центр ребра, якщо ребро довше порогу»). Такий підхід нестійкий до різних масштабів та форм елементів. Розроблений алгоритм є суто топологічним і не залежить від метричних властивостей елементів.

4. Комерційне та закрите програмне забезпечення. Найпотужніші системи (ANSYS, Abaqus, COMSOL) є комерційними і недоступні для студентів та дослідників

без інституційних ліцензій. Відкриті альтернативи суттєво поступаються за функціональністю в частині гексаедральних сіток.

5. Відсутність веб-доступу. Усі розглянуті системи є desktop-застосунками. Це унеможливлює спільну роботу над сіткою через браузер, розгортання в хмарному середовищі та інтеграцію в автоматизовані розрахункові конвеєри без повного встановлення пакету.

## 1.5 Висновки до розділу

Огляд восьми провідних систем генерації та уточнення скінченно-елементних сіток показав таке.

По-перше, жодна з розглянутих систем не реалізує шаблонного локального уточнення вже побудованої гексаедральної сітки. Уточнення або відсутнє взагалі, або зводиться до повторної генерації.

По-друге, доступні відкриті системи (Gmsh, FreeCAD, Salome) обмежені переважно тетраедральними сітками і не мають веб-інтерфейсу.

По-третє, комерційні системи (ANSYS, Abaqus, COMSOL) мають потужний функціонал, але є недоступними або недоступними для широкого кола користувачів через ціну ліцензій та відсутність відкритого API.

Таким чином, розробка відкритої веб-орієнтованої системи інтерактивного локального шаблонного уточнення гексаедральних сіток є актуальною задачею, що не має готового рішення серед існуючих інструментів.

# РОЗДІЛ 2
# АЛГОРИТМ ЛОКАЛЬНОГО АДАПТИВНОГО УТОЧНЕННЯ

Цей розділ є центральним у роботі і описує розроблений алгоритм локального адаптивного уточнення гексаедральних сіток. Алгоритм вирішує задачу: дано гексаедральну сітку та множину елементів, які користувач вибрав для подрібнення — побудувати нову сітку, в якій позначені елементи розбиті на кілька дрібніших без порушення конформності.

## 2.1 Концепція двох просторів представлення сітки

### 2.1.1 Мотивація

Перш ніж описувати алгоритм, важливо зрозуміти, як сітка зберігається в пам'яті. Класичний підхід — зберігати фізичні координати кожного вузла $(x, y, z)$ і список вузлів кожного елемента. Цей підхід простий, але має серйозний недолік: алгоритм уточнення стає залежним від конкретних чисел — координат. Якщо сітка деформована або масштабована, потрібно виправляти і алгоритм.

У розробленій системі використовується інший підхід: представлення сітки у двох незалежних просторах.

### 2.1.2 Ідеальний простір

Ідеальний простір — це чисто топологічне представлення сітки. Вузли характеризуються цілочисельними координатами $(i, j, k)$ у тривимірному випадку або $(i, j)$ у двовимірному. У цьому просторі всі елементи є правильними одиничними кубами або квадратами: не деформованими, не повернутими.

Ідеальний простір зберігає:

- зв'язність (які вузли утворюють кожен елемент),
- суміжність (які елементи мають спільні ребра або грані),
- ієрархію розбиттів (які елементи є дочірніми щодо яких).

Весь алгоритм уточнення виконується виключно в ідеальному просторі. Це забезпечує топологічну незалежність від геометрії.

### 2.1.3 Проміжний простір (напівфізичний)

Між ідеальним і фізичним простором існує проміжний рівень — стан сітки після уточнення топології, але до повного переходу в фізичну геометрію.

У проміжному просторі батьківські вузли зберігають цілочисельні координати, а форма батьківського елемента залишається кубом — як в ідеальному просторі.

Дочірні вузли, що з'явилися внаслідок уточнення, вже мають дійсні координати $(x, y, z)$, обчислені лінійною інтерполяцією між батьківськими вузлами. Проміжний простір є різновидом ідеального: батьківська структура сітки зберігає всі властивості ідеального простору (цілочисельні координати, кубічна форма елементів), але зона уточнення вже має геометрію у дійсних координатах.

### 2.1.4 Фізичний простір

Фізичний простір — це представлення сітки, де всі вузли мають дійсні координати $(x, y, z)$. Саме тут виконується лапласівське згладжування: оптимізація форми елементів у зоні уточнення шляхом переміщення вузлів. Згладжування переміщує всі вузли всередині зони уточнення — як дочірні, так і батьківські, що опинились у межах зони. Фіксованими залишаються лише вузли на границі зони (constraint nodes), що забезпечує неперервність сітки поза зоною. Критерій якості при згладжуванні — scaled Jacobian та aspect ratio кожного елемента зони.

Повний перехід до фізичного простору передбачає застосування ізопараметричного відображення до батьківських вузлів — переведення їх із цілочисельного домену в реальну геометрію через функції форми Q2-елемента. На поточному етапі розробки цей перехід не реалізований. Лапласівське згладжування реалізоване для двовимірного випадку; для тривимірного — заплановане як наступний крок розвитку.

### 2.1.5 Переваги трирівневого підходу

По-перше, відділення топології від геометрії: алгоритм уточнення (T1, T2, T4) повністю виконується в ідеальному просторі і не залежить від жодних координат. Це означає, що один алгоритм коректно працює для будь-якої форми вхідної сітки.

По-друге, ізольована оптимізація: лапласівське згладжування у фізичному просторі покращує якість елементів лише в межах зони уточнення, а вузли на границі зони залишаються фіксованими. Це запобігає деградації незачеплених частин сітки.

По-третє, тестованість: оскільки ідеальний простір є регулярним (всі елементи — однакові одиничні куби на цілочисельній решітці), результати уточнення легко верифікувати вручну — без залежності від конкретної геометрії.

По-четверте, масштабованість: перехід до повного фізичного простору може бути доданий пізніше без зміни алгоритму уточнення — лише як додатковий постпроцесинговий крок.

## 2.2 Загальна схема алгоритму уточнення

Алгоритм локального шаблонного уточнення гексаедральної сітки приймає на вхід сітку та множину елементів $\mathcal{M}$, вибраних користувачем для подрібнення. Результатом є конформна сітка: елементи із $\mathcal{M}$ рівномірно розбиті на $n_I \times n_J \times n_K$

дочірніх гексаедрів; сусідні елементи перехідної зони розбиваються за шаблоном T1, T2 або T4 залежно від кількості промаркованих вершин; решта сітки залишається незмінною.

Алгоритм складається з трьох послідовних проходів — по осях I, J та K. Кожен прохід виконує однакові кроки, але над різним напрямком і над поступово розширюваною множиною елементів:

1. Нумерація і формування пар. Розглянемо I-прохід (вісь X). Сітка складається з шарів — плоских зрізів перпендикулярних до осі X. Кожен такий шар містить $a \times b$ елементів, де $a$ — кількість елементів вздовж J (Y), $b$ — вздовж K (Z). Шари нумеруються: 1, 2, 3, 4, ... і групуються попарно: $[1,2]$, $[3,4]$, $[5,6]$, ... У кожній парі шарів елементи з'єднуються позиційно: елемент на позиції $(j,k)$ першого шару утворює пару з елементом на тій самій позиції $(j,k)$ другого шару. Таким чином одна пара шарів дає $a \times b$ пар елементів — по одній на кожну позицію в площині. Аналогічно для J- та K-проходів.

   Непарна кількість шарів. Якщо загальна кількість шарів непарна, до кінця додається уявний (фіктивний) шар без жодного елемента з $\mathcal{M}$. Він утворює пару з останнім реальним шаром. Оскільки фіктивний шар порожній, активність цієї пари визначається виключно останнім реальним шаром: якщо в ньому є елементи з $\mathcal{M}$ — вузли зовнішньої грані геометрії отримують позначку.

2. Маркування вузлів. Для кожної пари $[2k,\ 2k{+}1]$: якщо хоча б один з двох гексаедрів належить до $\mathcal{M}$ — чотири вузли їхньої спільної грані отримують позначку. Пари, де жоден елемент не в $\mathcal{M}$, пропускаються.

   б Унікальність вузлів і поширення позначок. Кожен вузол сітки є унікальним об'єктом — дублювання заборонено. Один і той самий вузол може одночасно належати кільком гексаедрам: батьківським, дочірнім або сусіднім. Коли 4 вузли спільної грані отримують позначку, ці самі вузли вже належать і до сусідніх гексаедрів. Для гексаедра з пари — це 4 промаркованих вузли (T4). Для сусіда по осі J або K — лише 2 спільних вузли на одному ребрі (T2). Для діагонального сусіда — 1 вузол (T1). Таким чином маркування автоматично поширюється на сусідів — без жодних додаткових операцій.

3. Застосування шаблону. Кожен гексаедр із хоча б одним позначеним вузлом розбивається. Шаблон визначається кількістю промаркованих вершин елемента:

| Позначених вузлів | Шаблон | Примітка |
|---|---|---|
| 1 | T1 | |
| 2 | T2 | |
| 4 | T4 | |
| 8 | — | елемент пропускається (шаблон не застосовується) |

4. Розширення зони. Щойно прохід завершено, до $\mathcal{M}$ додаються всі щойно розбиті елементи і їхні сусіди по наступних осях — щоб наступний прохід міг узгодити їхні грані.

Завдяки поступовому розширенню $\mathcal{M}$ між проходами кожен наступний прохід охоплює ширшу зону і коректно будує перехідні елементи у всіх трьох напрямках.

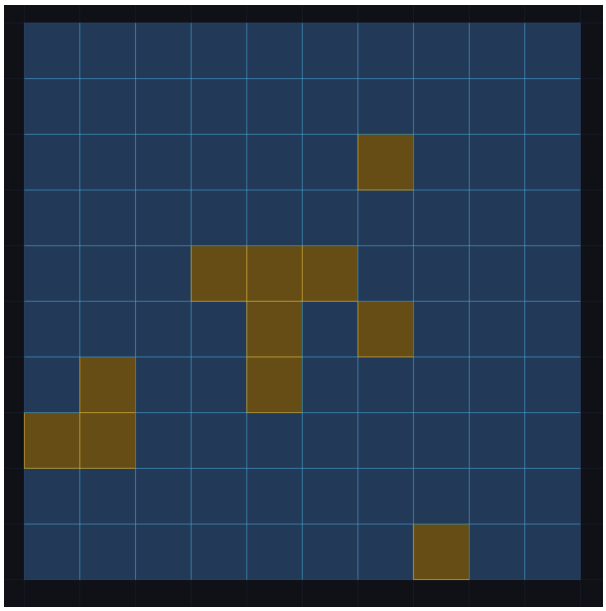


Рис. 2: Вибір елементів для уточнення

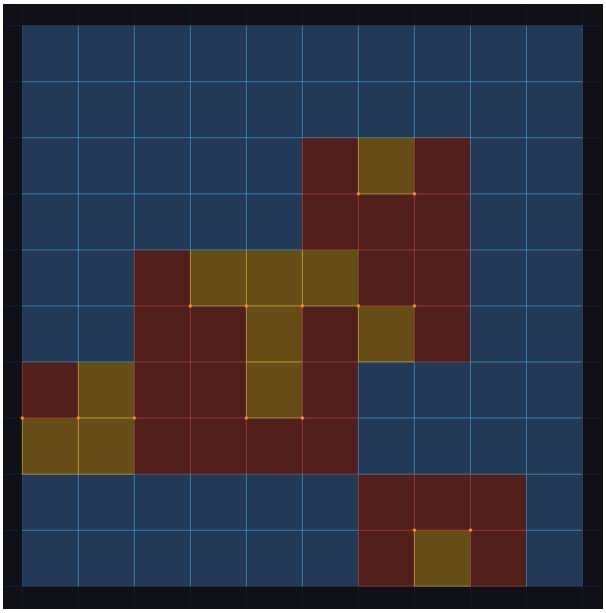


Рис. 3: Крок 1: маркування вузлів вибраних елементів (червоний — позначені)

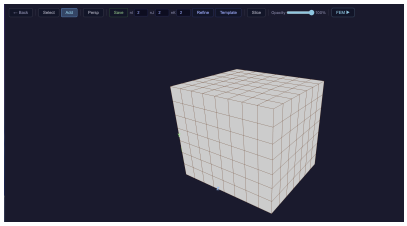


Рис. 4: Початкова 3D-сітка

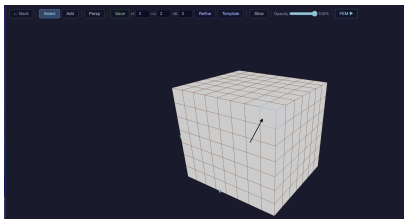


Рис. 5: Вибір елемента для уточнення



Рис. 6: Задання параметрів 5×5×5

### 2.2.1 Нумерація шарів елементів (BuildChains)

Чому 2-уточнення потребує пар шарів. Ключова властивість 2-уточнення (де кожне ребро ділиться на два) полягає в тому, що перехідна зона між уточненою і незміненою частинами сітки завжди охоплює рівно два сусідніх шари елементів — не один і не три, а саме два. Це принципова відмінність від 3-уточнення, де перехідний шар може бути одним. Пояснення просте: при поділі ребра навпіл нова точка з'являється між двома сусідами, і щоб шаблон переходу замкнувся коректно, потрібен «простір» з

обох сторін — ліворуч і праворуч від нової точки. Тому перед кожним кроком уточнення алгоритм визначає, які елементи є сусідами-парами вздовж кожної з трьох осей.

Що таке шар і ланцюжок. Вздовж кожної осі сітка природно розпадається на шари елементів. Один шар — це максимальна послідовність листових (нерозбитих) сусідніх елементів, що йдуть підряд у напрямку осі. Такий шар і називається ланцюжком. Ланцюжок переривається там, де немає сусіда (межа сітки або розрив від попереднього уточнення). Аналогічно визначаються ланцюжки вздовж осей J та K.

Глобальна нумерація і парування. Усі шари-ланцюжки вздовж осі нумеруються послідовно: 0, 1, 2, 3, 4, 5, ... При цьому нумерація є глобальною — лічильник не скидається при розриві ланцюжка, а продовжується далі. Це гарантує, що пари завжди складаються з елементів із сусідніми номерами: шари 0 і 1 утворюють пару 0, шари 2 і 3 — пару 1, і так далі. Якщо загальна кількість шарів непарна, останній шар залишається без пари і пропускається в поточному проході. Процедура виконується окремо для кожної з трьох осей.

Навіщо це потрібно при уточненні. Коли алгоритм обробляє I-напрямок, він розглядає кожну пару сусідніх шарів разом. Якщо хоча б один елемент із пари потрапив під уточнення — вузли на спільній грані між шарами маркуються і шаблони переходу застосовуються узгоджено з обох боків. Саме це гарантує, що нові вузли з двох сторін грані збігаються і не утворюють висячих вузлів. Без такого узгодженого парування виникали б або висячі вузли, або необхідність каскадно поширювати уточнення на сусідні елементи.

## 2.3 Шаблони розбиття

Шаблон розбиття визначається кількістю та розташуванням промаркованих вершин гексаедра. Використовуються три шаблони: T1, T2 та T4.

### 2.3.1 Шаблон T1 — одна промаркована вершина

Шаблон застосовується при $n = 2$ (поділ ребра навпіл). Рівно одна з 8 вершин гексаедра позначена — назвемо її $V_i$. З позначеної вершини виходять 3 ребра, 3 грані та 1 просторова діагональ. Алгоритм вставляє нові вузли в середині кожного з цих відрізків:

- 3 вузли на ребрах від $V_i$ (по осях X, Y, Z),
- 3 вузли на гранях від $V_i$ (середини граней XY, XZ, YZ),
- 1 вузол у центрі гексаедра (середина просторової діагоналі від $V_i$).

Разом 7 нових вузлів. Вони утворюють «внутрішню копію» гексаедра масштабу $\frac{1}{2}$, вкладену у кут $V_i$. Гексаедр ділиться на 4 дочірніх: кутовий куб (при $V_i$) і три трапецієподібних гексаедри, що покривають три зовнішні грані, протилежні до $V_i$.

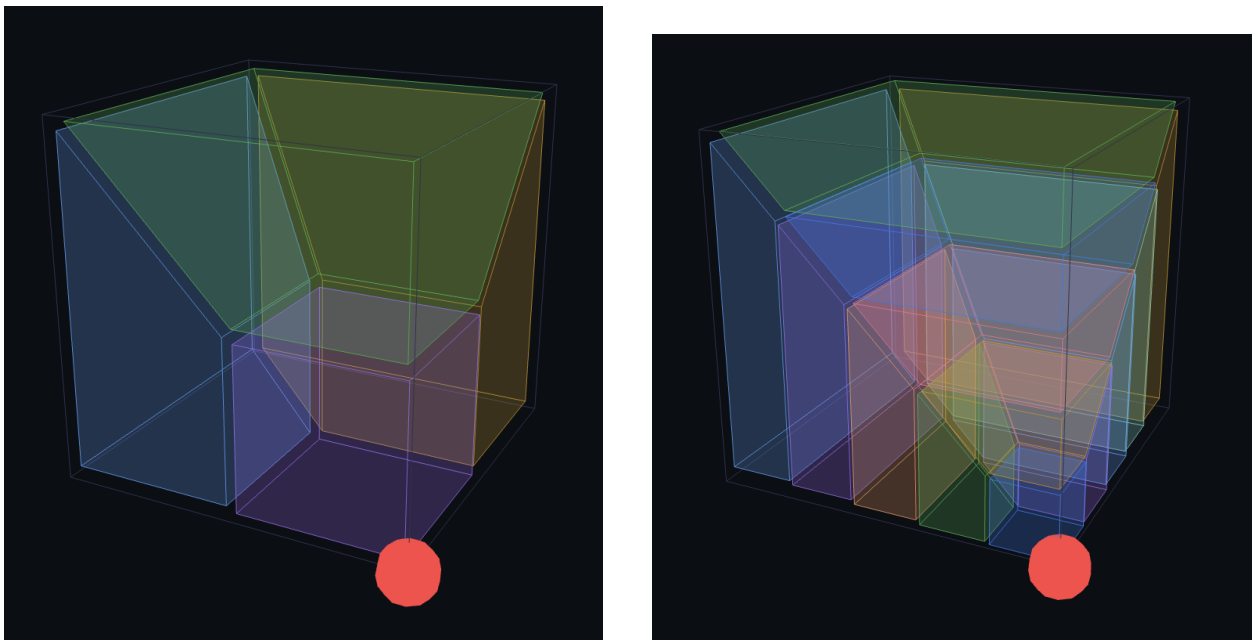


Рис. 7: Шаблон T1: базовий $n = 2$ (ліворуч) та анізотропний $n = 5$ (праворуч)

### 2.3.2 Шаблон T2 — два промаркованих вузли на одному ребрі

Шаблон застосовується при $n = 2$. Два з 8 вузлів гексаедра позначені, і вони утворюють одне ребро — назвемо його $V_a$–$V_b$. Це ребро лежить на двох перпендикулярних гранях гексаедра.

На кожній з цих двох граней виконується 2D T1-шаблон: вставляються середини двох ребер грані від позначеного кута і центр грані. Разом — 6 нових вузлів (4 серединні точки ребер + 2 центри граней). Потім відповідні 2D-частини двох граней з'єднуються між собою в 3D:

- кутовий гексаедр — містить позначене ребро $V_a$–$V_b$,
- два крилових гексаедри — розходяться від кутового по двох напрямках, перпендикулярних до ребра.

Гексаедр ділиться на 3 дочірніх.

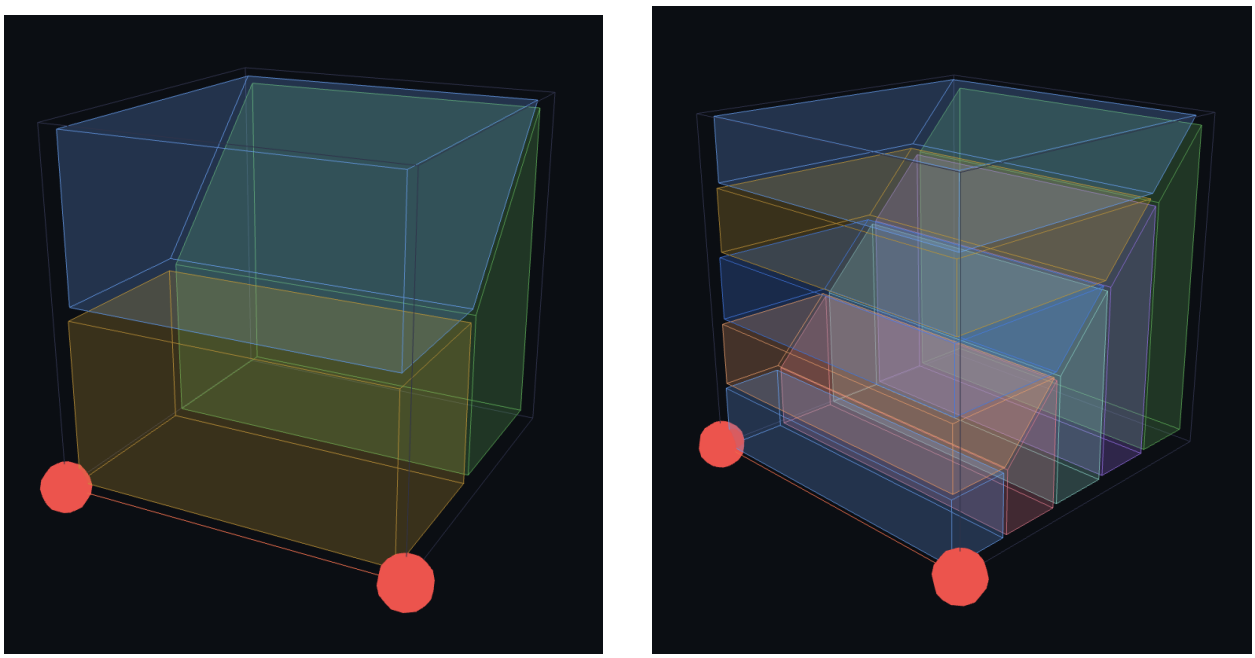


Рис. 8: Шаблон T2: базовий $n = 2$ (ліворуч) та анізотропний $n = 5$ (праворуч)

### 2.3.3 Шаблон T4 — чотири промаркованих вузли на одній грані

Шаблон застосовується при $n = 2$. Усі 4 кути однієї грані позначені. Це найпростіший і найякісніший шаблон: посередині гексаедра паралельно до позначеної грані вставляється одна серединна площина. Гексаедр ділиться на 2 рівних дочірніх — два прямокутних паралелепіпеди однакового розміру. Scaled Jacobian дочірніх дорівнює Scaled Jacobian батька.

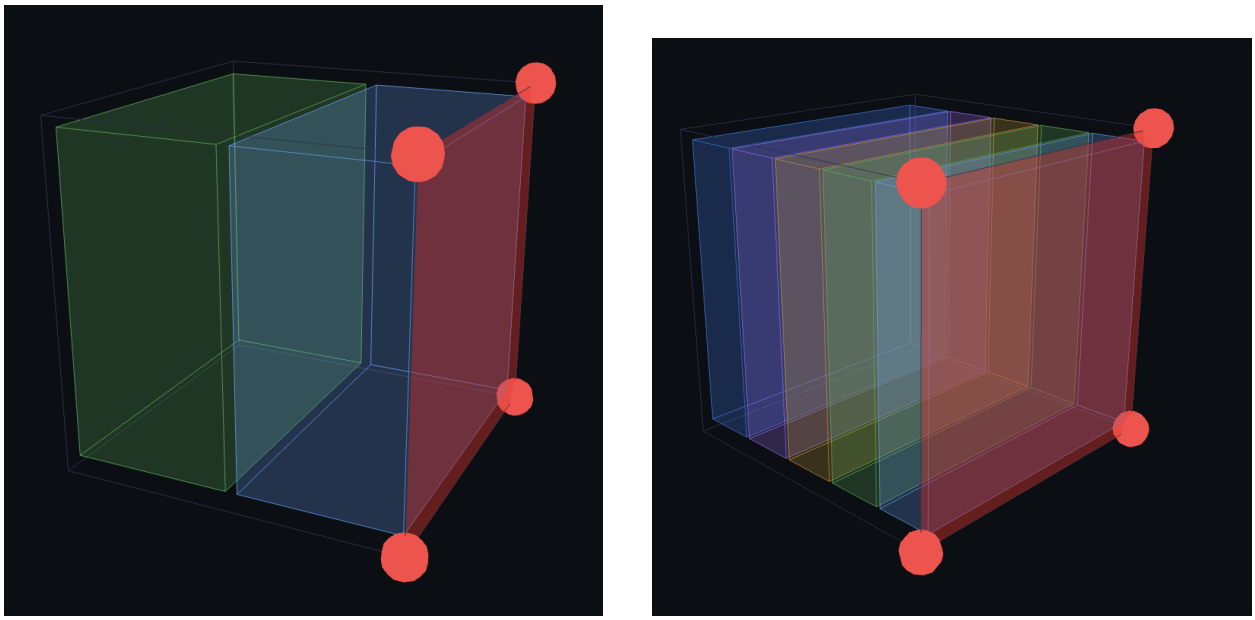


Рис. 9: Шаблон T4: базовий $n = 2$ (ліворуч) та анізотропний $n = 5$ (праворуч)

Табл. 2: Шаблони розбиття гексаедра

| Шаблон | Умова (к-ть і розташування позначених вузлів) | Дочірніх |
|---|---|---|
| T1 | 1 вузол (будь-який кут) | 4 |
| T2 | 2 вузли, що утворюють ребро | 3 |
| T4 | 4 вузли — повна грань | 2 |

## 2.4 Маркування вузлів та парування шарів

### 2.4.1 Чому потрібні пари шарів

При бінарному поділі вздовж осі новий вузол з'являється на грані між двома сусідами. Цей вузол породжує дві «відкриті точки» — одну на правій грані лівого елемента, одну на лівій грані правого. Щоб сітка залишилась конформною, ці дві точки мусять замкнутись: або дійти до межі геометрії, або з'єднатися між собою. Пара шарів $[2k,\ 2k{+}1]$ і є саме таким замиканням: права відкрита точка шару $2k$ і ліва відкрита точка шару $2k{+}1$ — це той самий спільний вузол на їхній спільній грані. Цикл замкнувся всередині пари, не поширюючись на решту сітки.

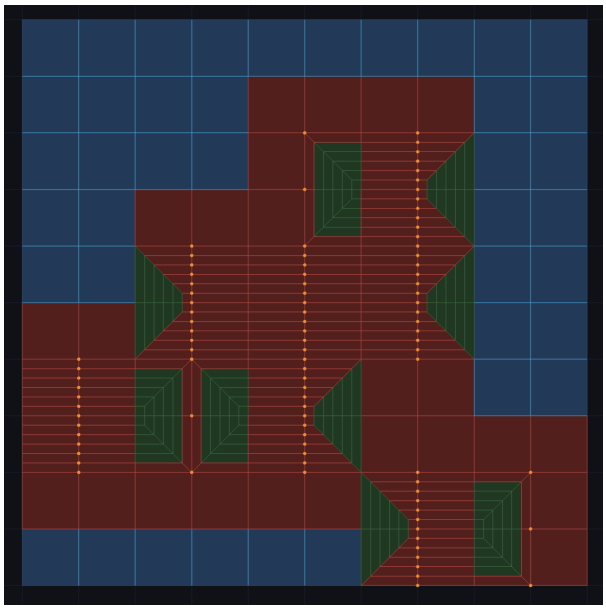


Рис. 10: Застосування шаблонів T1 і T2 (крок 2: розширення $\mathcal{M}$ + маркування)

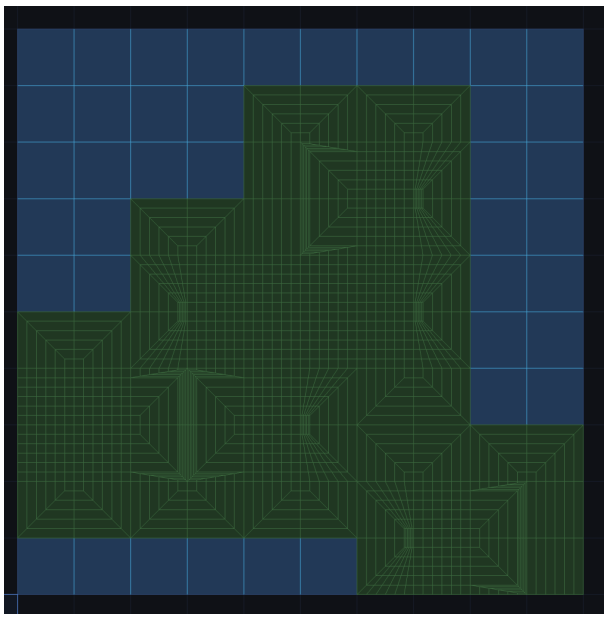


Рис. 11: Результуюча сітка після застосування шаблонів

## 2.5 Три проходи: I → J → K

Ключова ідея трипрохідного алгоритму: задачу тривимірного уточнення розкладають на три незалежних одновимірних задачі — по осях I, J та K. Проходи виконуються послідовно, і кожен наступний прохід вже знає, де попередній ввів нові вузли. Саме це знання дозволяє підібрати правильний шаблон розбиття без висячих вузлів.

Ще одна принципова деталь: множина оброблюваних елементів $\mathcal{M}$ розширюється між проходами. Після I-проходу до $\mathcal{M}$ додаються всі елементи, що стали сусідами вздовж J та K; після J-проходу — сусіди вздовж K. Це гарантує, що жоден внутрішній елемент не залишиться з неузгодженими гранями.

### 2.6.1 I-прохід

На початку I-проходу $\mathcal{M}$ містить лише ті елементи, які користувач вибрав для уточнення — вихідна зона уточнення без будь-яких розширень.

Алгоритм виконує такі кроки:

1. Нумерація ланцюжків по осі I. Функція `BuildChains` починає з першого гексаедра в рядку і крок за кроком переходить до наступного сусіда вздовж осі I. Кожному листовому (нерозбитому) гексаедру присвоюється глобальний порядковий номер: $0, 1, 2, \ldots$ Два сусідніх елементи з індексами $2k$ та $2k+1$ утворюють пару.

2. Маркування граней пари. Для кожної пари, в якій хоча б один елемент $\in \mathcal{M}$, маркуються чотири вузли на спільній внутрішній грані між $2k$ та $2k+1$ (а також на зовнішніх гранях крайніх елементів, якщо пара на межі сітки).

3. Застосування шаблону. Шаблон визначається кількістю промаркованих вузлів кожного елемента:

   - елементи пари ($2k$ і $2k{+}1$) мають усі 4 вузли спільної грані позначені — застосовується T4 (2 дочірніх);
   - сусіди по осях J та K мають 2 спільних вузли (спільне ребро) — застосовується T2 (3 дочірніх);
   - діагональні сусіди мають 1 спільний вузол — застосовується T1 (4 дочірніх).

Після I-проходу вихідна зона уточнення розбита по осі I. Сусіди розбитих елементів по осях J та K поділяють з ними один або два спільних вузли — ці вузли вже промарковані, тому сусіди потребують обробки на наступних проходах.

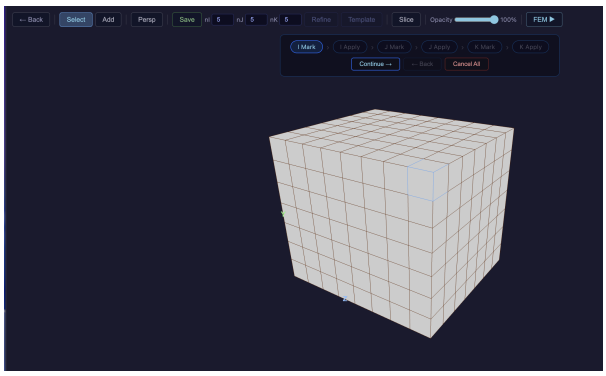


Рис. 12: I-прохід: початок маркування

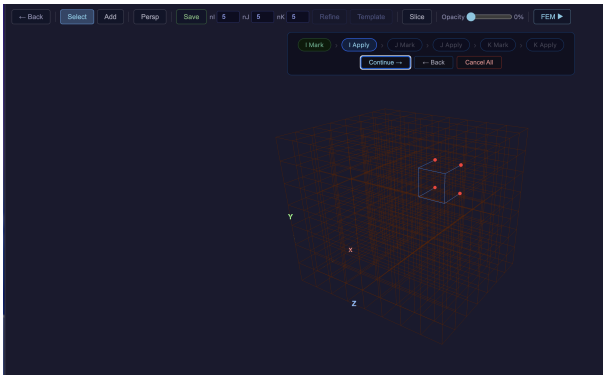


Рис. 13: I-прохід: позначені вузли

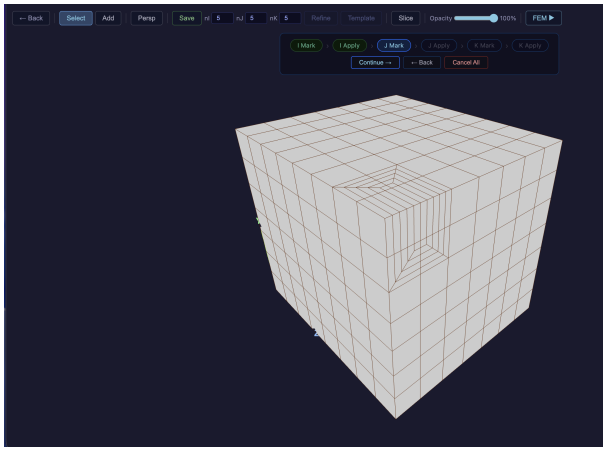


Рис. 14: Результат I-проходу

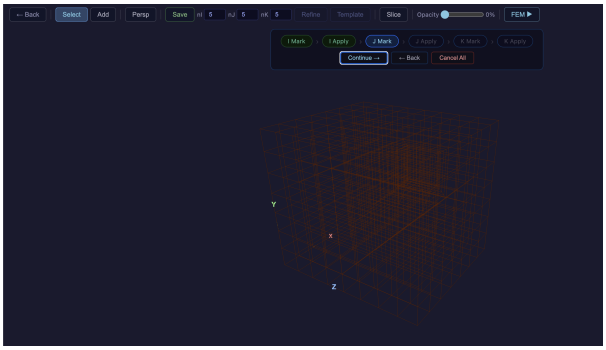


Рис. 15: Шаблони після I-проходу

### 2.6.2 Розширення $\mathcal{M}$ після I-проходу

Перед J-проходом до $\mathcal{M}$ додаються всі елементи, що мають хоча б один промаркований вузол і ще не входять до множини. Це J- та K-сусіди I-розбитих елементів: вони вже мають 1 або 2 промарковані вузли від спільних ребер і будуть оброблені J-проходом.

### 2.6.3 J-прохід

J-прохід обробляє розширену $\mathcal{M}$ вздовж осі J. Логіка повністю ідентична I-проходу.

1. Нумерація ланцюжків по осі J. `BuildChains` виконується по зв'язках `NeighborBottom`/`NeighborTop`, утворюючи пари вздовж J.

2. Маркування граней пари по J. Для пар, що перетинаються з $\mathcal{M}$, маркуються вузли на спільній внутрішній грані.

3. Вибір шаблону. Шаблон визначається кількістю промаркованих вузлів: 1 → T1, 2 → T2, 4 → T4, 8 → пропустити. Якщо елемент вже розбитий (не є листовим) — шаблон застосовується безпосередньо до його дочірніх елементів за тим самим правилом залежно від кількості їхніх промаркованих вузлів. В окремих випадках для покращення якості перехідної зони додатково застосовується алгоритм Node Propagation: від зовнішнього промаркованого вузла лінійною інтерполяцією обчислюються позиції вузлів на кожному рівні ієрархії; для кожної такої позиції знаходиться дочірній елемент, що її містить, і до нього додається мітка — після чого шаблон (T1, T2 або T4) застосовується вже до цих дочірніх елементів.

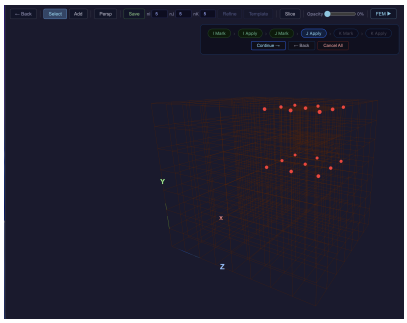


Рис. 16: J-прохід: позначені вузли

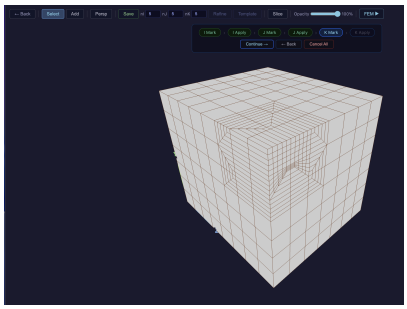


Рис. 17: Результат J-проходу

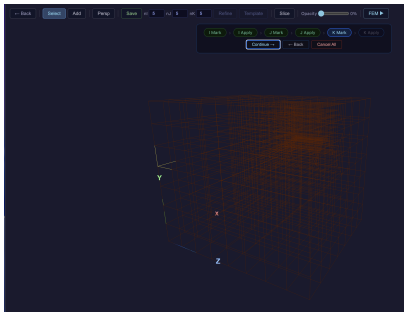


Рис. 18: Шаблони після J-проходу

### 2.6.4 Розширення $\mathcal{M}$ після J-проходу

Аналогічно до кроку 2.6.2: до $\mathcal{M}$ додаються K-сусіди елементів, розбитих або оброблених на J-проході, якщо вони ще не входять до множини і мають промарковані вузли.

### 2.6.5 K-прохід

K-прохід отримує повністю розширену $\mathcal{M}$ і виконується аналогічно до I- та J-проходів.

1. Нумерація ланцюжків по осі K. `BuildChains` обходить зв'язки `NeighborFront`/`NeighborBack`.

2. Маркування граней пари по K. Вузли на спільній K-грані пари.

3. Вибір шаблону. Те саме правило: кількість промаркованих вузлів визначає шаблон ($1 \to$ T1, $2 \to$ T2, $4 \to$ T4). Якщо елемент вже розбитий — шаблон застосовується до його дочірніх елементів за тим самим правилом залежно від кількості їхніх промаркованих вузлів.

Після K-проходу кожен елемент у розширеній $\mathcal{M}$ розбитий відповідним шаблоном, усі спільні грані між сусідніми елементами мають однакові вузли — висячих вузлів немає.

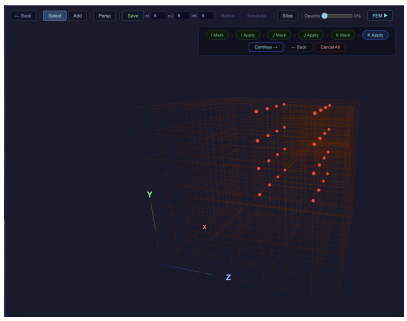


Рис. 19: K-прохід: позначені вузли

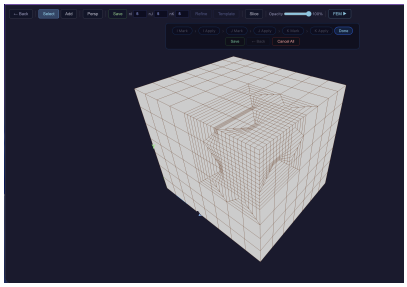


Рис. 20: Результат K-проходу

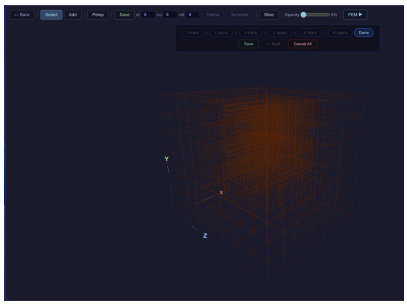


Рис. 21: Шаблони після K-проходу

## 2.6.6 Node Propagation — класифікація кейсів

Node Propagation застосовується виключно до нелистових елементів — тих, що вже були розбиті у попередньому проході (мають дочірні). Коли J- або K-прохід намагається нанести мітки на зовнішні вузли такого батька, безпосереднє застосування шаблону до батька неможливе — потрібно пропагувати мітки вглиб ієрархії до реальних листових дочірніх.

Загальний принцип (єдиний для T1 і T2 батька). Нехай батьківський елемент вже розбитий і має $n_1$ шарів дочірніх уздовж відповідної осі. Зовнішня мітка прийшла на вузол $V_k$ батька, а $V_c$ — кутовий вузол батька, від якого відраховується глибина ієрархії.

Алгоритм проходить всі $n_1$ рівнів вглиб:

$$\mathbf{p}(\ell) = \mathrm{Lerp}(V_c, V_k, \tfrac{n_1-\ell}{n_1}), \qquad \ell = 0, 1, \ldots, n_1 - 1.$$

При $\ell = 0$ отримуємо саму зовнішню мітку $V_k$; при $\ell = n_1 - 1$ — позицію безпосередньо поруч із кутом. На кожному рівні $\ell$ знаходиться дочірній елемент, що містить точку $\mathbf{p}(\ell)$, і цей дочірній отримує відповідну мітку. Після обходу всіх рівнів до кожного позначеного дочірнього застосовується шаблон за стандартним правилом: 1 мітка $\to$ T1, 2 на одному ребрі $\to$ T2, 4 на одній грані $\to$ T4.

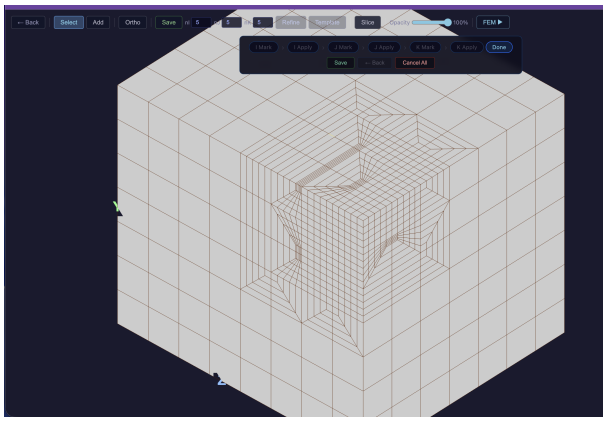


Рис. 22: Фінальна сітка: ортогональний вид

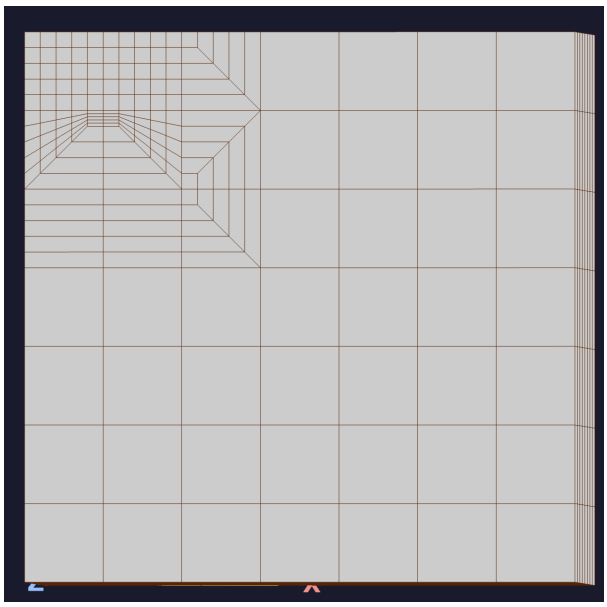


Рис. 23: Вид вздовж осі X

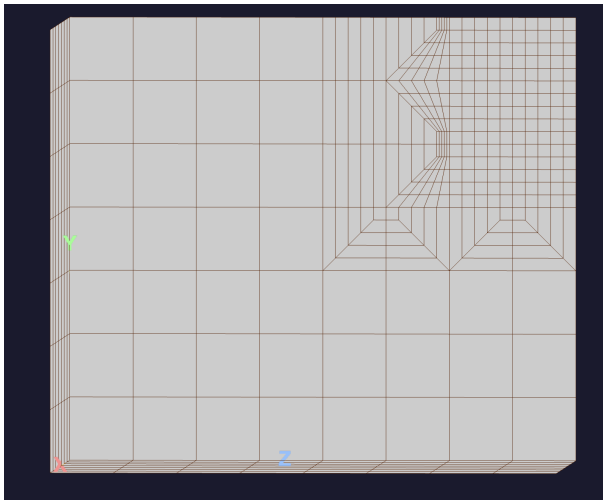


Рис. 24: Вид вздовж осі Y

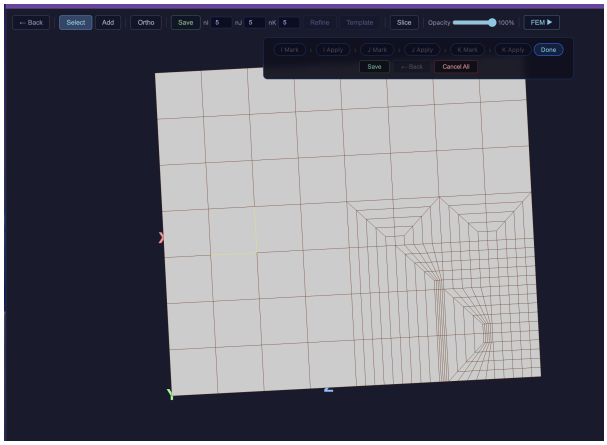


Рис. 25: Вид вздовж осі Z

Цей механізм однаковий незалежно від того, чи батько поділений шаблоном T1, чи T2, і незалежно від глибини вкладеності ієрархії.

## Кейси для T1-батька

T1-батько розбитий по куту $V_0$. Його 6 граней поділяються на два типи:

- Брудні грані (3 штуки, що примикають до $V_0$): Bottom $\{v_0, v_1, v_4, v_5\}$, Front $\{v_0, v_1, v_2, v_3\}$, Left $\{v_0, v_3, v_4, v_7\}$. На ребрах цих граней вже є вузли рівнів level$[1 \ldots \mathrm{n}-1]$; нові мітки потрапляють до дочірніх безпосередньо, без propagation.
- Чисті грані (3 штуки, що не примикають до $V_0$): Right $\{v_1, v_2, v_5, v_6\}$, Top $\{v_2, v_3, v_6, v_7\}$, Back $\{v_4, v_5, v_6, v_7\}$. На них немає жодного вставленого вузла — мітки потребують propagation.

Після T1 з $n = 2$ батько має 4 дочірніх: X-hex, Y-hex, Z-hex (шар 0) та кутовий куб (corner cube).

T1-батько, 1 мітка ($V_i$ позначений).

Вузол $V_i$ знаходиться в одному з 8 напрямків-променів від $V_0$:

Табл. 3: Кейси propagation від T1-батька — 1 мітка

| Позиція $V_i$ | Напрямок від $V_0$ | Результат (n=2) |
|---|---|---|
| $V_i = V_0$ | кут T1 (0 осей) | тільки кутовий куб → T1 |
| $V_i = v_1, v_3$ або $v_4$ | ребро (1 вісь, X/Y/Z) | 1 face-hex + кутовий куб → T1 |
| $V_i = v_2, v_5$ або $v_7$ | діагональ грані (2 осі) | 2 face-hexes + кутовий куб → T1 |
| $V_i = V_d = v_6$ | просторова діагональ (3 осі) | всі 3 face-hexes + кутовий куб → T1 |

Паттерн: кутовий куб завжди отримує T1 (він містить level[1][j] для будь-якого $j$). Кількість face-hexes, що отримують T1, дорівнює кількості осей у напрямку $V_i$ від $V_0$.

T1-батько, 2 мітки ($V_a$ і $V_b$).

Кейс 2v-dirty ($V_a = V_0$ або $V_b = V_0$): один вузол є самим кутом T1. $V_0$ маркується лише в кутовому кубі. Другий вузол поширюється за правилом 1v (propagation або пряме маркування залежно від типу грані). Кутовий куб отримує 2 мітки → T2; відповідний face-hex отримує 1 мітку → T1.

Кейс 2v-clean ($V_a \neq V_0$ і $V_b \neq V_0$, обидва на чистій грані): мітки переносяться на level[1].

- Якщо $V_a$, $V_b$ утворюють ребро гексаедра — відповідні дочірні отримують 2 мітки на ребрі → T2. Приклад: ребро $v_1$–$v_2$ на Right face → X-hex: T2, Y-hex: T1, кутовий куб: T2.
- Якщо $V_a$, $V_b$ утворюють діагональ грані — §4.9 автоматично доповнює до 4 вузлів → переходить до кейсу 4v-clean.

T1-батько, 4 мітки ($V_a, V_b, V_c, V_d$).

Кейс 4v-dirty (хоча б один з чотирьох $= V_0$): брудна грань. $V_0$ потрапляє до кутового куба (1 мітка → T1), осьові сусіди — до відповідних face-hexes (по 1 мітці → T1). Результат: T1 + 1–2 × T1 + none (залежно від розташування інших трьох вузлів).

Кейс 4v-clean (жоден з чотирьох $\neq V_0$): чиста грань, вимагає propagation. Всі 4 мітки переносяться з level[0] на level[1] тієї самої грані.

Загалом: 2×T4 + 2×T2 — проти T4 + 2×T2 + none без propagation. Кутовий куб переходить із відсутності шаблону до T4 (SJ → 1.0 замість SJ ≈ 0.2–0.3).

Табл. 4: Propagation T1-батька — 4v на чистій Right-грані ($v_1, v_2, v_5, v_6$)

| Дочірній | Мітки level[1] | Шаблон |
|---|---|---|
| X-hex | r[1], r[2], r[5], r[6] (всі 4 на одній грані) | T4 |
| Y-hex | r[2], r[6] (ребро [1,5]) | T2 |
| Z-hex | r[5], r[6] (ребро [1,2]) | T2 |
| Кутовий куб | r[1], r[2], r[5], r[6] (грань {1,2,5,6}) | T4 |

### Кейси для T2-батька

T2-батько розбитий по ребру $\{V_0, V_1\}$, яке паралельне до осі X (наприклад ребро $v_0$–$v_1$). «Труба» T2 простягається у Z-напрямку; батько має $n-1$ frame-hexes та 1 edge-hex (ребровий куб).

T2-батько, 1 мітка ($V_i$).

Кейс $V_i \in \{V_0, V_1\}$ (один з кутів ребра T2): $V_i$ знаходиться на «брудній» грані (містить ребро T2) — пряме маркування. Edge-hex отримує 1 мітку → T1.

Кейс $V_i \notin \{V_0, V_1\}$ (на «чистій» грані): мітка переноситься на level[1] через propagation. Відповідні дочірні отримують T1.

T2-батько, 2 мітки ($V_a$, $V_b$).

Кейс 1: $V_a \in \{V_0, V_1\}$ або $V_b \in \{V_0, V_1\}$ (один вузол ребра T2 серед маркованих):

- Вузол-кут ребра → пряме маркування edge-hex (T1).
- Інший вузол → propagation або пряме маркування залежно від типу своєї грані; відповідний face-hex отримує T1 або T2.

Кейс 2: $V_a \notin \{V_0, V_1\}$ і $V_b \notin \{V_0, V_1\}$ (обидва на чистих гранях): мітки переносяться на level[1] через propagation.

- Якщо утворюють ребро → T2 у відповідних дочірніх.
- Якщо утворюють діагональ грані → §4.9 доповнює до 4 вузлів, переходить до кейсу 4v.

T2-батько, 4 мітки ($V_a, V_b, V_c, V_d$) — повна грань.

Якщо 4 мітки утворюють повну грань, паралельну ребру T2: propagation переносить їх на відповідні внутрішні вузли дочірніх — результат 2×T4 (проти T4 + деформований T2 без propagation, Owen et al. рис. 14).

Зведена таблиця якості.

Для зони переходу: $SJ \geq 0.5$ з propagation проти $SJ \approx 0.3$ без нього. Після Taubin-згладжування (розд. 3): $SJ \to 0.6$–$0.8$.

Анізотропна гнучкість. Батьківський елемент (T1 або T2) може бути поділений з параметром $n_1$ уздовж однієї осі, а наступні проходи алгоритму для його дочірніх

Табл. 5: Вплив Node Propagation на якість перехідної зони

| Конфігурація | Без propagation | З propagation |
|---|---|---|
| T1-батько, 4v (чиста грань) | T4 + 2×T2 + none | 2×T4 + 2×T2 |
| T1-батько, 2v (ребро на чистій грані) | T2 + T1 + none ×2 | 2×T2 + T1 |
| T1-батько, 1v | T1 + none | T1 + T1 (або T2) |
| T2-батько, 4v (паралельна грань) | T4 + деф. T2 | 2×T4 |

елементів застосовують власний параметр $n_2$, незалежно вибраний для кожного напрямку. Таким чином система є повністю адаптивною: кожен рівень ієрархії вузлів отримує власне анізотропне розбиття, що дозволяє точно відтворювати геометрію з різним масштабом деталізації вздовж різних осей.

## Ілюстрації кейсів Node Propagation

Нижче наведено вибіркові знімки інтерактивного візуалізатора для кейсів Node Propagation. Кожне зображення показує стан «До» та «Після».

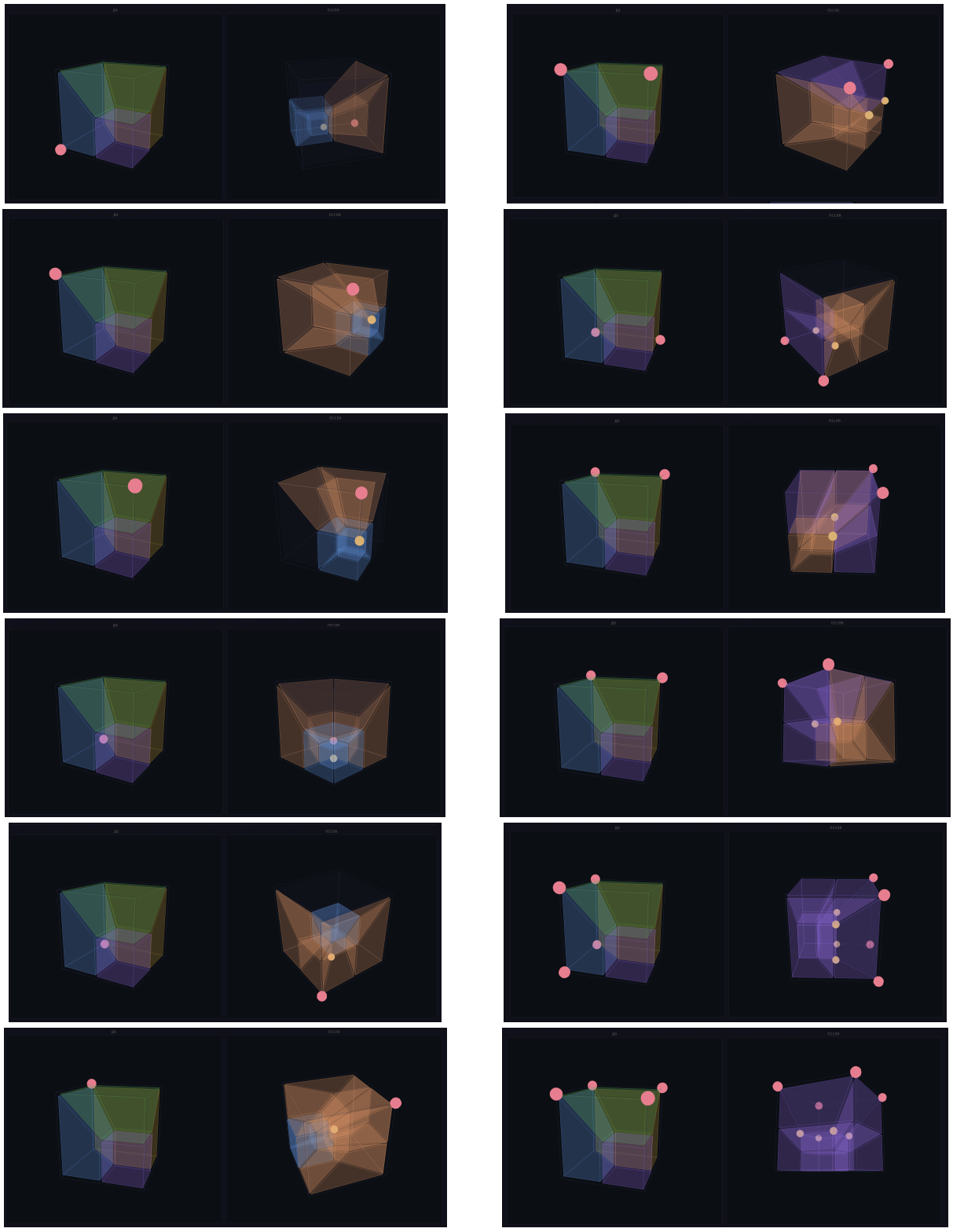


Рис. 26: Кейси Node Propagation (вибірка), сторінка 1

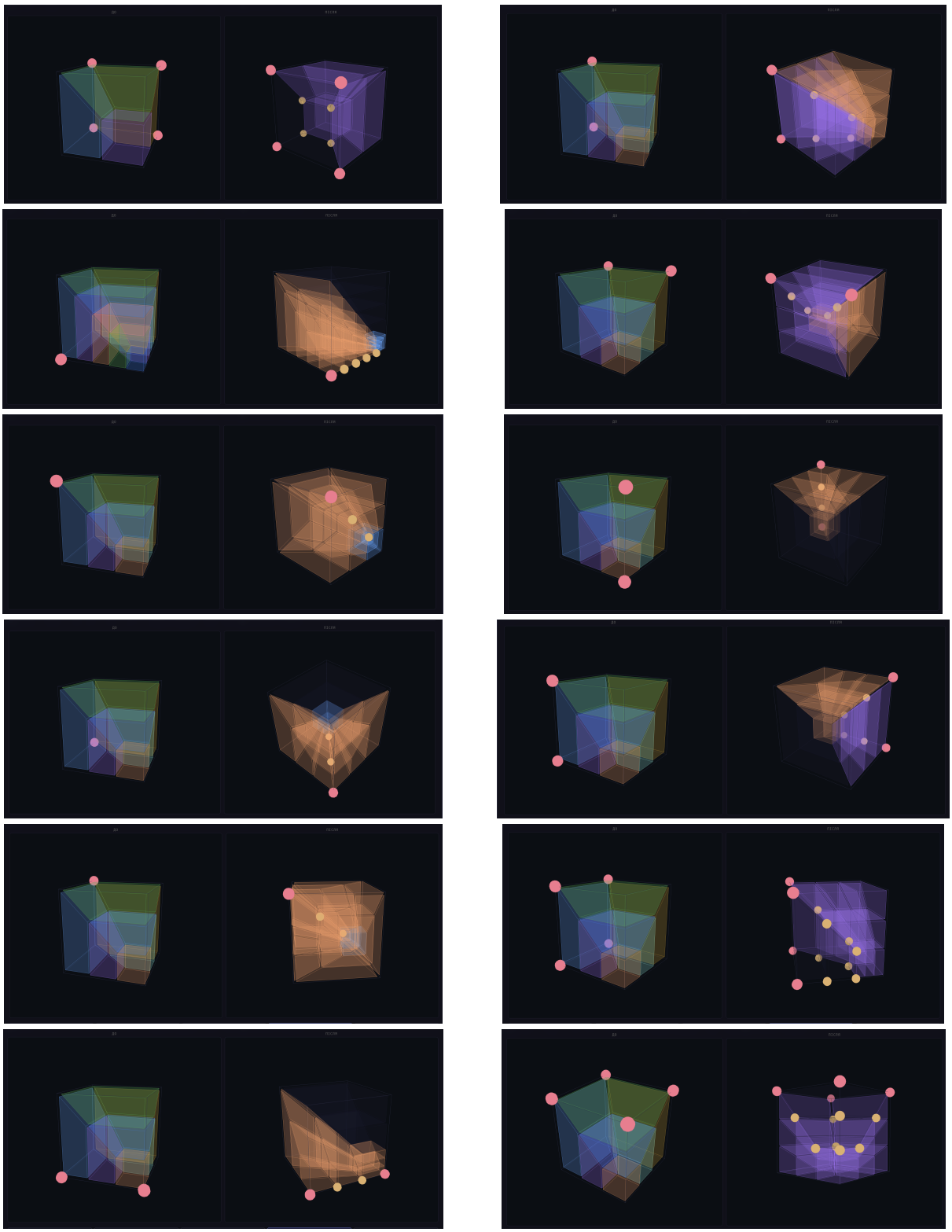


Рис. 27: Кейси Node Propagation (вибірка), сторінка 2

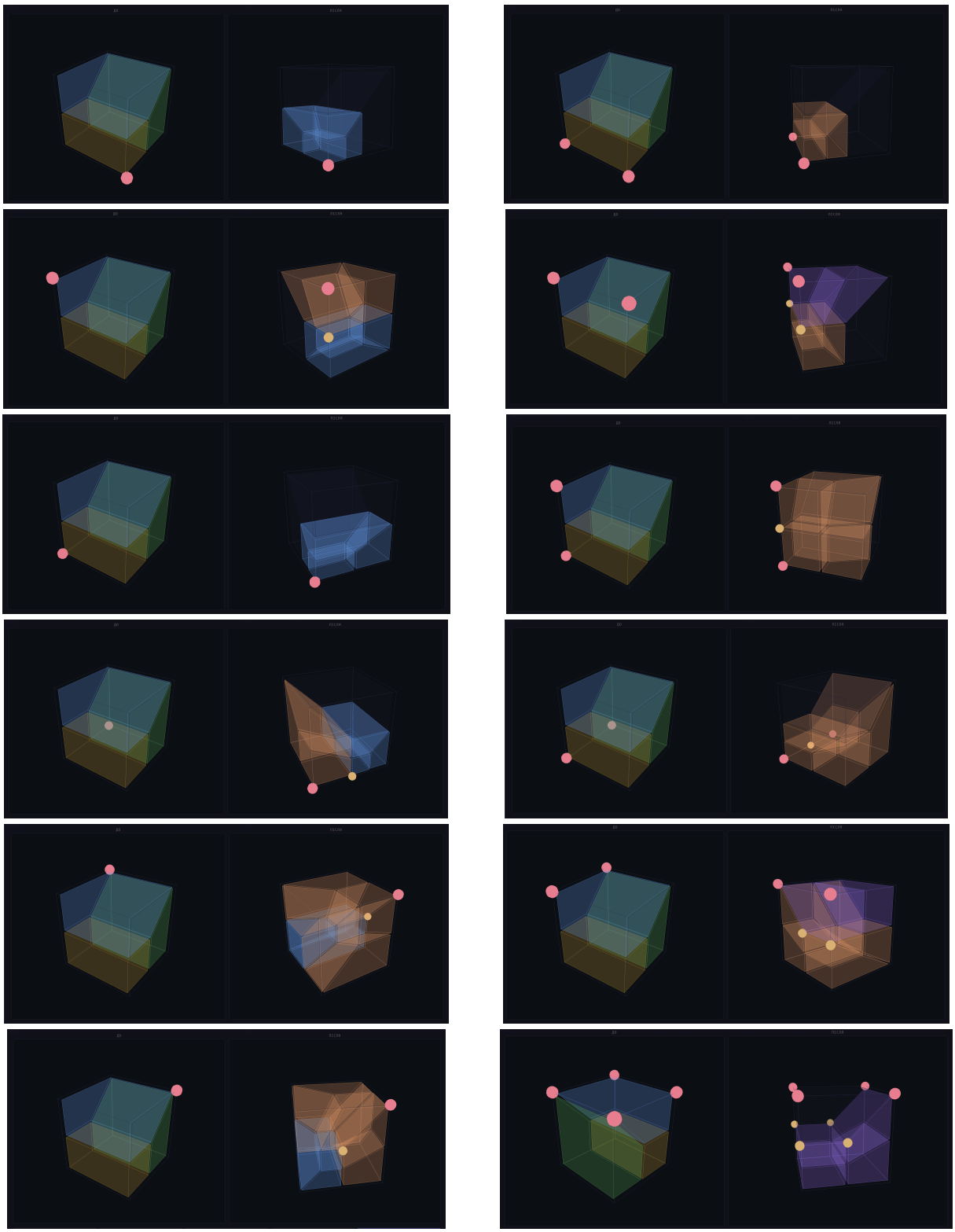


Рис. 28: Кейси Node Propagation (вибірка), сторінка 3

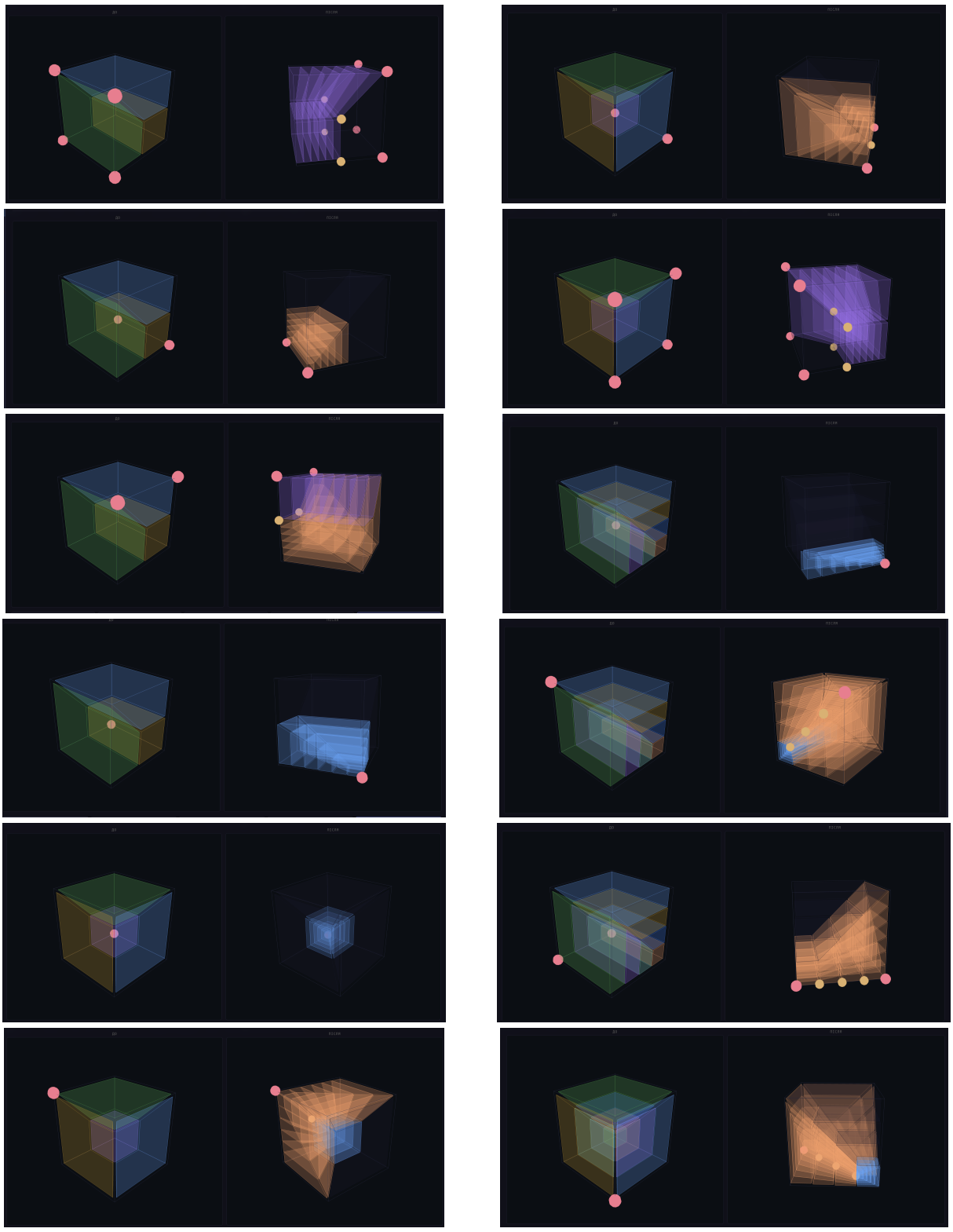


Рис. 29: Кейси Node Propagation (вибірка), сторінка 4

## 2.7 Анізотропне уточнення

Параметри $n_I$, $n_J$, $n_K$ є незалежними і можуть приймати різні значення. Це дозволяє реалізовувати анізотропне уточнення — розбиття елемента різний раз уздовж різних осей.

Наприклад:

- $n_I = 4$, $n_J = 1$, $n_K = 1$: розбиття тільки по I у 4 рази. Корисно для сіток з виразно витягнутими елементами по I (наприклад, балка).
- $n_I = 2$, $n_J = 2$, $n_K = 1$: розбиття в площині IJ у 4 рази. Корисно для плитоподібних конструкцій.
- $n_I = 2$, $n_J = 2$, $n_K = 2$: рівномірне 8-кратне розбиття (класичне).

Анізотропне уточнення особливо корисне в граничних шарах (тонкі елементи нормально до поверхні) або при моделюванні тонкостінних конструкцій.

## 2.8 Транзакційний механізм сесії уточнення

Для підтримки можливості скасування незавершених операцій у системі реалізовано транзакційний механізм сесії уточнення.

Інтерфейс `IRefinementSession` надає три операції:

BeginAsync(meshId) — розпочинає сесію. Поточний стан сітки серіалізується у JSON і зберігається як знімок.

CancelAsync(meshId) — скасовує сесію. Сітка відновлюється зі знімка. Позначається як змінена для наступного автозбереження.

CommitAsync(meshId) — підтверджує сесію. Знімок видаляється, зміни записуються через механізм автозбереження.

Система підтримує не більше однієї активної сесії для кожної сітки. Якщо при виклику `BeginAsync` виявляється активна сесія, вона автоматично скасовується перед початком нової.

Транзакційний механізм є критичним для надійної роботи системи в умовах мережевих збоїв або раптового закриття браузера: при наступному підключенні незавершена сесія буде виявлена та скасована.

## 2.9 Складність алгоритму

Введемо позначення:

- $N$ — загальна кількість листових елементів сітки,
- $M$ — кількість елементів, вибраних для подрібнення ($M \leq N$),
- $L$ — середня довжина ланцюжка по одній осі,
- $n_I, n_J, n_K$ — параметри кількості поділів по кожній осі.

BuildChains. Алгоритм обходить кожен листовий елемент рівно один раз по кожній осі, призначаючи йому індекс у ланцюжку. Складність: $O(N)$ на один виклик, $O(3N) = O(N)$ для трьох осей.

Маркування вузлів (MarkPass). Для кожної пари $[2k, 2k+1]$ перевіряється приналежність до $\mathcal{M}$ і маркуються 4 вузли спільної грані. Оскільки вузли унікальні й маркування поширюється автоматично через спільні вузли — додаткових обходів сусідів не потрібно. Складність: $O(N)$.

I-прохід (ApplyPass). Для кожного елемента $\mathcal{M}$ визначається шаблон ($O(1)$) і застосовується розбиття ($O(n_I)$ нових вузлів і елементів). Разом: $O(M \cdot n_I)$.

Розширення $\mathcal{M}$ (ExpandMarkedSet). Обхід листових нащадків щойно розбитих елементів: $O(M \cdot n_I)$ у найгіршому випадку.

J- та K-проходи. Аналогічно I-проходу, але $\mathcal{M}$ вже розширена — може містити до $O(M \cdot L)$ елементів (сусіди вздовж ланцюжка). Складність кожного проходу: $O(M \cdot L \cdot n_J)$ та $O(M \cdot L \cdot n_K)$ відповідно.

Node Propagation. Виконується лише для нелистових елементів, яких при одному рівні уточнення $O(M)$. Для кожного — лінійний обхід рівнів ієрархії глибиною $O(\log n)$. Загалом: $O(M \cdot \log n)$, що не домінує.

Загальна складність одного кроку уточнення:

$$T_{\text{refine}} = O\big(N + M \cdot L \cdot \max(n_I, n_J, n_K)\big) \tag{2}$$

Розглянемо окремий практичний випадок — рівномірна сітка: це сітка, де елементи розташовані у вигляді тривимірного прямокутного масиву $p \times q \times r$ і кожен бік приблизно однаковий, тобто $p \approx q \approx r \approx N^{1/3}$ (тобто кубічний корінь з $N$; наприклад, для $N = 1000$ елементів у сітці $10 \times 10 \times 10$ маємо $N^{1/3} = 10$). Тоді довжина одного ланцюжка вздовж будь-якої осі — це кількість елементів уздовж тієї осі: $L \approx N^{1/3}$. Підставляємо у загальну формулу при $M \ll N$:

$$T_{\text{refine}} \approx O\big(N + M \cdot N^{1/3} \cdot n_{\max}\big)$$

Перший доданок $O(N)$ — це обхід усієї сітки функцією `BuildChains`. Другий доданок $O(M \cdot N^{1/3} \cdot n_{\max})$ — це власне розбиття зони уточнення. Оскільки $M \ll N$, другий доданок значно менший за перший, і загальна складність залишається близькою до лінійної відносно $N$.

Практичний висновок: час відгуку алгоритму залежить переважно від розміру зони уточнення $M$, а не від розміру всієї сітки — що і робить його придатним для інтерактивного використання.

## 2.11 Висновки до розділу

Розроблений алгоритм локального адаптивного уточнення гексаедральних сіток базується на п'яти ключових ідеях.

1. Два простори: алгоритм виконується в ідеальному (топологічному) просторі, незалежно від фізичної геометрії елементів.

2. Ланцюжкова нумерація: BuildChains будує глобальну нумерацію ланцюжків, що забезпечує детерміноване виконання.

3. Node Propagation: без цього механізму ізольований T1-шаблон на крайовому вузлі породжує трапецієподібні дочірні елементи з гострими кутами — Scaled Jacobian може падати нижче 0.3. Node Propagation поширює маркування вглиб ієрархії, перетворюючи T1 $\rightarrow$ T2 (Scaled Jacobian $\geq 0.5$) або T2 $\rightarrow$ T4 (Scaled Jacobian $\approx$ SJ батька), що суттєво покращує співвідношення сторін та кутів перехідних елементів.

4. Анізотропні параметри: $n_I$, $n_J$, $n_K$ можуть відрізнятися, що підтримує анізотропне уточнення.

5. Транзакційна сесія: механізм begin/cancel/commit захищає цілісність сітки при будь-яких збоях.

Складність є близькою до лінійної при типових сценаріях.

# РОЗДІЛ 3
# МЕТОДИ ПОКРАЩЕННЯ ЯКОСТІ СІТКИ ПІСЛЯ УТОЧНЕННЯ

Після локального адаптивного уточнення геометрична якість елементів поблизу зон розбиття може погіршитися. Нові вузли вводяться в рівномірних позиціях в ідеальному просторі, але в фізичному просторі (особливо для деформованих або криволінійних сіток) ці позиції можуть утворювати не зовсім правильні квадрати або куби. Цей розділ описує методи, що застосовуються для покращення якості сітки після уточнення.

## 3.1 Що таке якість скінченно-елементної сітки

### 3.1.1 Вплив якості на МСЕ

Якість елементів безпосередньо впливає на точність МСЕ-розв'язку. Два конкретних ефекти:

Погана обумовленість: якщо елемент деформований (дуже витягнутий або «скошений»), то якобіан відображення у деяких точках Гаусса стає близьким до нуля або від'ємним. Це призводить до поганої обумовленості локальної матриці жорсткості $K^{(e)}$, що вносить значні числові помилки.

Інвертовані елементи: якщо якобіан $\det(\mathbf{J}) < 0$, елемент «перевернутий» — його внутрішні та зовнішні боки поміняні місцями. Інтеграл матриці жорсткості для такого елемента не має фізичного змісту. МСЕ-система рівнянь стане некоректною.

### 3.1.2 Основні метрики якості

Для оцінки якості використовуються такі числові характеристики.

Масштабований якобіан (Scaled Jacobian, SJ) — головна метрика якості гексаедрального елемента. Вона вимірює, наскільки кути елемента відрізняються від прямих ($90^\circ$).

Фізична ідея: у кожному з 8 кутів гексаедра сходяться 3 ребра. Якщо ці три ребра утворюють між собою прямі кути (як у кубі) — якість ідеальна. Якщо кути скошені — якість падає.

Для кожного кута $k$ гексаедра обчислюється локальне значення:

$$SJ_k = \frac{\det(\mathbf{e}_{1,k}, \mathbf{e}_{2,k}, \mathbf{e}_{3,k})}{|\mathbf{e}_{1,k}| \cdot |\mathbf{e}_{2,k}| \cdot |\mathbf{e}_{3,k}|} \tag{3}$$

де $\mathbf{e}_{1,k}$, $\mathbf{e}_{2,k}$, $\mathbf{e}_{3,k}$ — три вектори, що виходять з кута $k$ вздовж трьох ребер гексаедра. Чисельник — об'єм паралелепіпеда, утвореного цими трьома векторами. Знаменник — добуток їхніх довжин, тобто об'єм того самого паралелепіпеда, якби всі кути були рівно $90^\circ$. Відношення показує, наскільки реальний кут «прямий»: при ідеальних $90^\circ$ чисельник = знаменник, тому $SJ_k = 1$.

Глобальний $SJ$ елемента — мінімум за всіма 8 кутами: $SJ = \min_k SJ_k$.

Інтерпретація значень:

- $SJ = 1.0$ — ідеальний куб, всі кути рівно $90°$;
- $SJ \in [0.5, 1.0)$ — добра якість, прийнятна для більшості МСЕ-задач;
- $SJ \in [0.2, 0.5)$ — перехідна зона, числова точність знижується;
- $SJ \in [0.0, 0.2)$ — погана якість, розрахунок може давати помилки;
- $SJ < 0$ — елемент «перевернутий» (інвертований), розрахунок неможливий.

Співвідношення сторін (Aspect Ratio, AR) — відношення найдовшого ребра до найкоротшого серед усіх 12 ребер гексаедра:

$$AR = \frac{l_{\max}}{l_{\min}} \tag{4}$$

При $AR = 1$ — куб (всі ребра однакові). При $AR \gg 1$ — елемент сильно витягнутий, що погіршує точність МСЕ.

Мінімальний кут (лише для 2D квадрилатералів): мінімальний внутрішній кут між двома суміжними ребрами квада. У 3D цю роль виконує $SJ$.

Рекомендовані значення для МСЕ-розрахунків: $SJ \geq 0.5$ (бажане), $SJ \geq 0.2$ (абсолютний мінімум), $AR \leq 5$.

## 3.2 Аналіз методів згладжування та вибір підходу

Після локального згущення сітки перехідна зона містить елементи зі скошеними кутами та нерівномірним розподілом розмірів, що знижує точність МСЕ-розрахунку. У цьому розділі розглядаються можливі методи згладжування, аналізуються їхні переваги і недоліки, та обґрунтовується вибір методу для майбутньої реалізації.

### 3.2.1 Лапласівське згладжування

Позначення. Кожен вузол сітки — це точка у тривимірному просторі з координатами $(x, y, z)$. Позначимо координати $i$-го вузла як $\mathbf{p}_i$ (жирний шрифт означає, що це вектор з трьох чисел: $\mathbf{p}_i = (x_i, y_i, z_i)$).

Сусіди вузла $\mathbf{p}_i$ — усі вузли, з'єднані з ним ребром сітки. Позначимо їх $\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_m$, де $m$ — кількість сусідів (для внутрішнього вузла гексаедральної сітки типово $m = 3$ або 6).

Формула оновлення. На кожній ітерації кожен внутрішній вузол переміщується у бік середнього арифметичного позицій своїх сусідів:

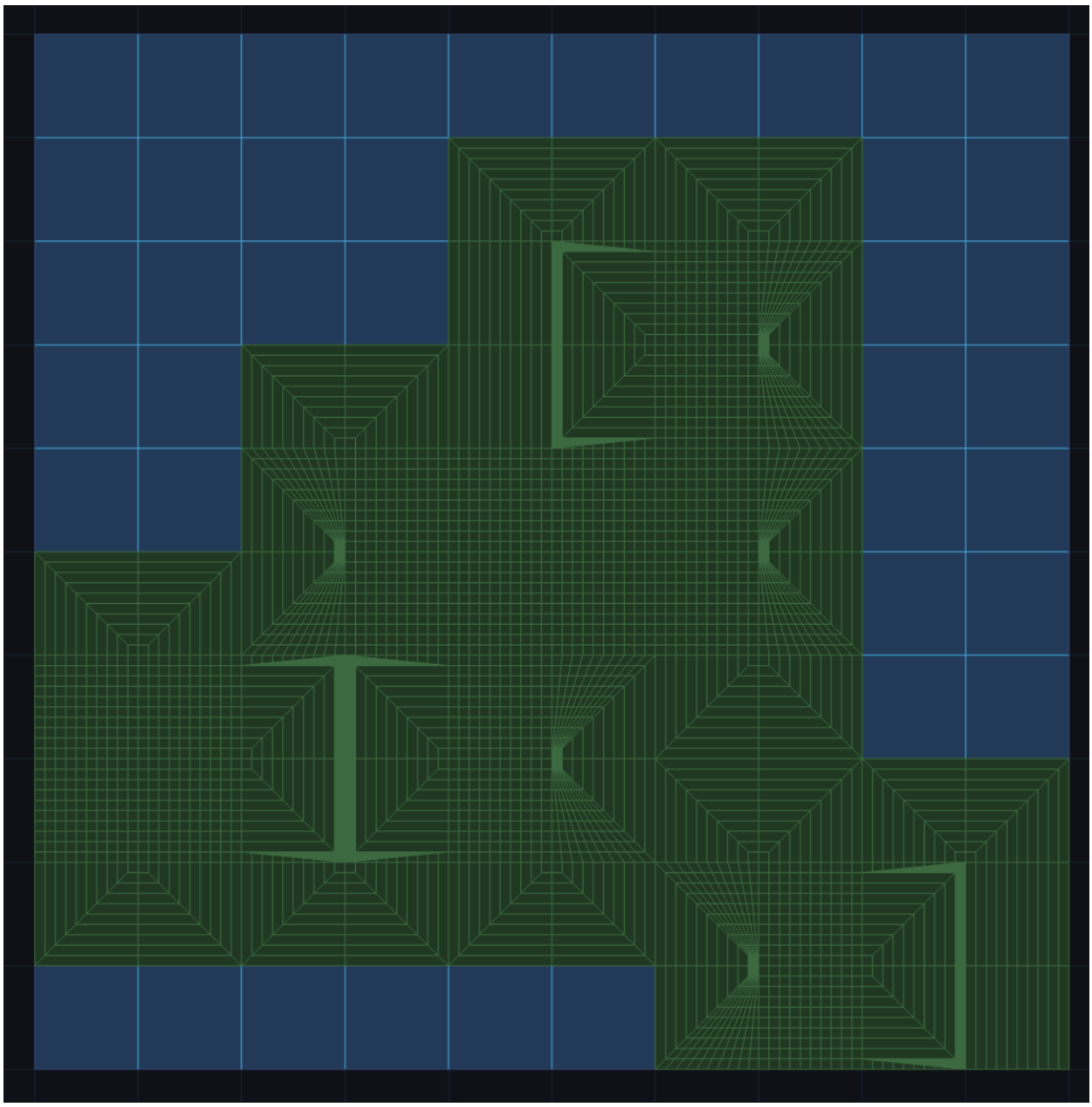


Рис. 30: Сітка 10×10 після уточнення: низька якість кутів та погане співвідношення сторін без згладжування

$$\mathbf{p}_i^{\text{new}} = \mathbf{p}_i + \lambda \underbrace{\left( \frac{1}{m} \sum_{j=1}^{m} \mathbf{p}_j - \mathbf{p}_i \right)}_{\text{вектор від } \mathbf{p}_i \text{ до середнього сусідів}} \tag{5}$$

Що означає кожна частина формули:

- $\frac{1}{m} \sum_{j=1}^{m} \mathbf{p}_j$ — середнє арифметичне координат усіх $m$ сусідів (середня точка між ними);
- $\frac{1}{m} \sum_j \mathbf{p}_j - \mathbf{p}_i$ — вектор, що вказує від поточного вузла до цієї середньої точки;
- $\lambda \in (0, 1]$ — коефіцієнт релаксації, визначає яку частку цього вектора пройде вузол

за одну ітерацію (при $\lambda = 0.5$ вузол рухається рівно до половини відстані);

- $\mathbf{p}_i^{\text{new}}$ — нова позиція вузла після кроку.

Що таке усадка (shrinkage) і чому вона виникає. Проблема усадки добре відома в літературі: при повторному застосуванні лапласіану поверхня сітки поступово «стягується» до центру.

Причина — геометрична. Розглянемо кільце вузлів, що апроксимує коло (рис. 31). Кожен вузол $\mathbf{p}_i$ стоїть на дузі кола. Його два сусіди $\mathbf{p}_{i-1}$ та $\mathbf{p}_{i+1}$ теж на дузі, але хорда між ними коротша за дугу — тобто середнє $\frac{1}{2}(\mathbf{p}_{i-1} + \mathbf{p}_{i+1})$ знаходиться всередині кола, ближче до центру. Лапласіан переміщує $\mathbf{p}_i$ у цьому напрямку — ближче до центра.

Після 10 ітерацій у числовому прикладі (рис. 31) радіус полігону зменшився з $r = 1.00$ до $r \approx 0.60$ — усадка 40%. Форма геометричного об'єкта суттєво спотворилась.

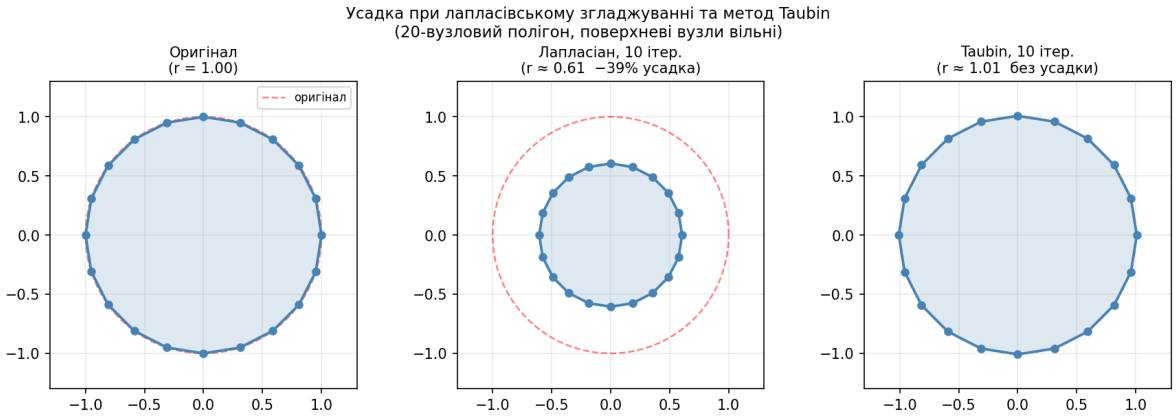



Рис. 31: Усадка при лапласівському згладжуванні vs. метод Taubin. 20-вузловий полігон (апроксимація кола), вузли вільні. Лапласіан за 10 ітерацій зменшив радіус до 0.60 (-40%); Taubin зберіг форму ($r \approx 1.01$).

Vollmer et al. показали, що навіть із фіксованими вузлами на зовнішній межі внутрішні вузли стягуються до центра, і запропонували «повернення» до попередньої позиції. Taubin вирішив задачу інакше — через зворотній крок з від'ємним параметром (розд. 3.2.2).

Переваги: простота реалізації, складність $O(N)$ на ітерацію.

Недоліки:

- Усадка: повторне застосування деформує геометрію об'єкта.
- Нерівномірність: покращує кути, але не вирівнює розміри елементів — поряд залишаються великі і дрібні гексаедри.

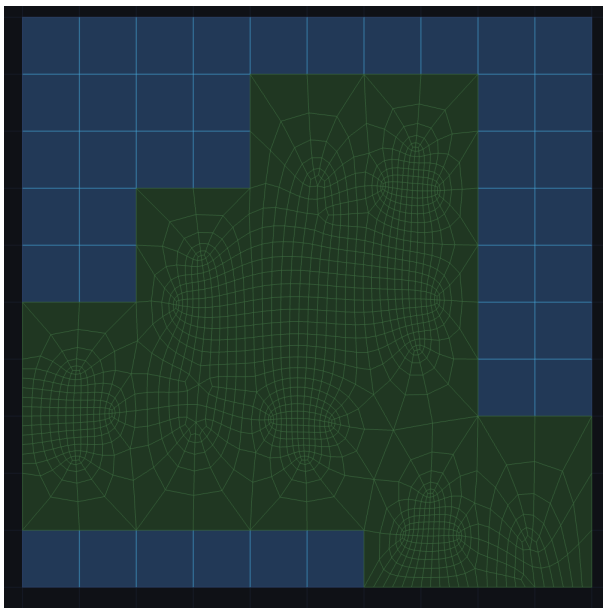


Рис. 32: Лапласівське згладжування (мала сітка)

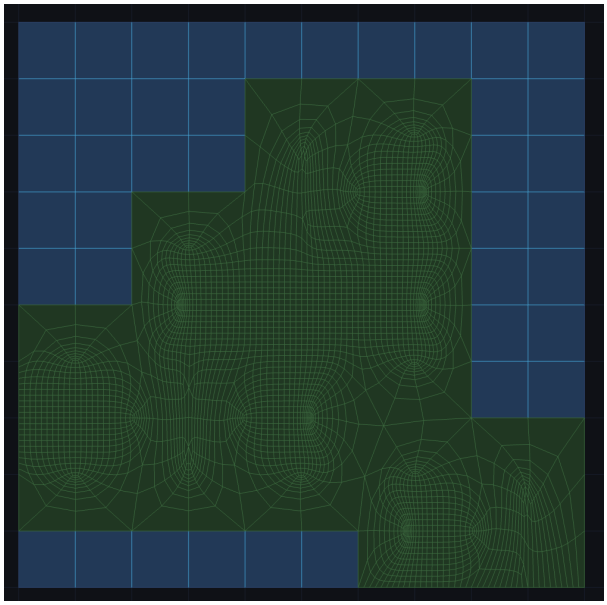


Рис. 33: Лапласівське згладжування (10×10)

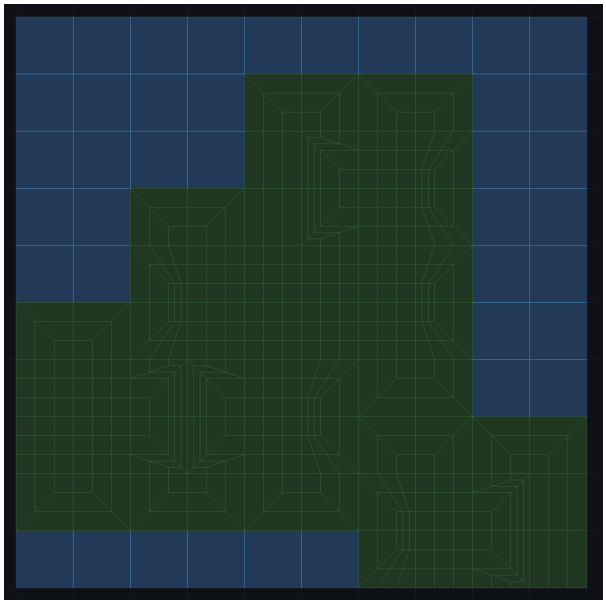


Рис. 34: Сітка 3×3 до лапласівського згладжування

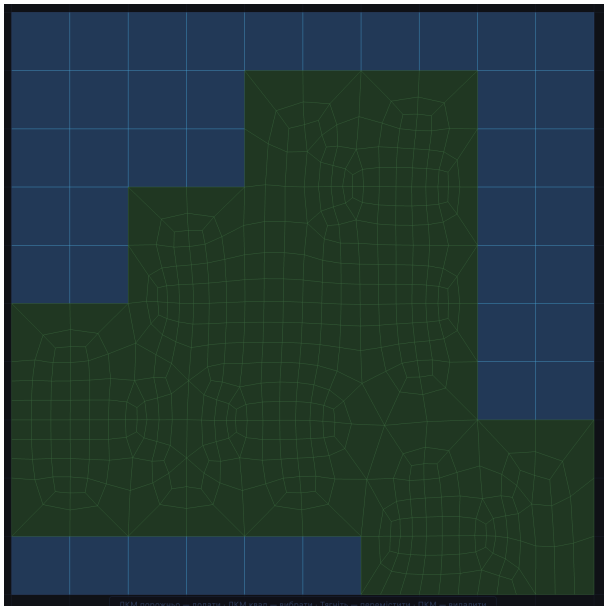


Рис. 35: Сітка 3×3 після лапласівського згладжування

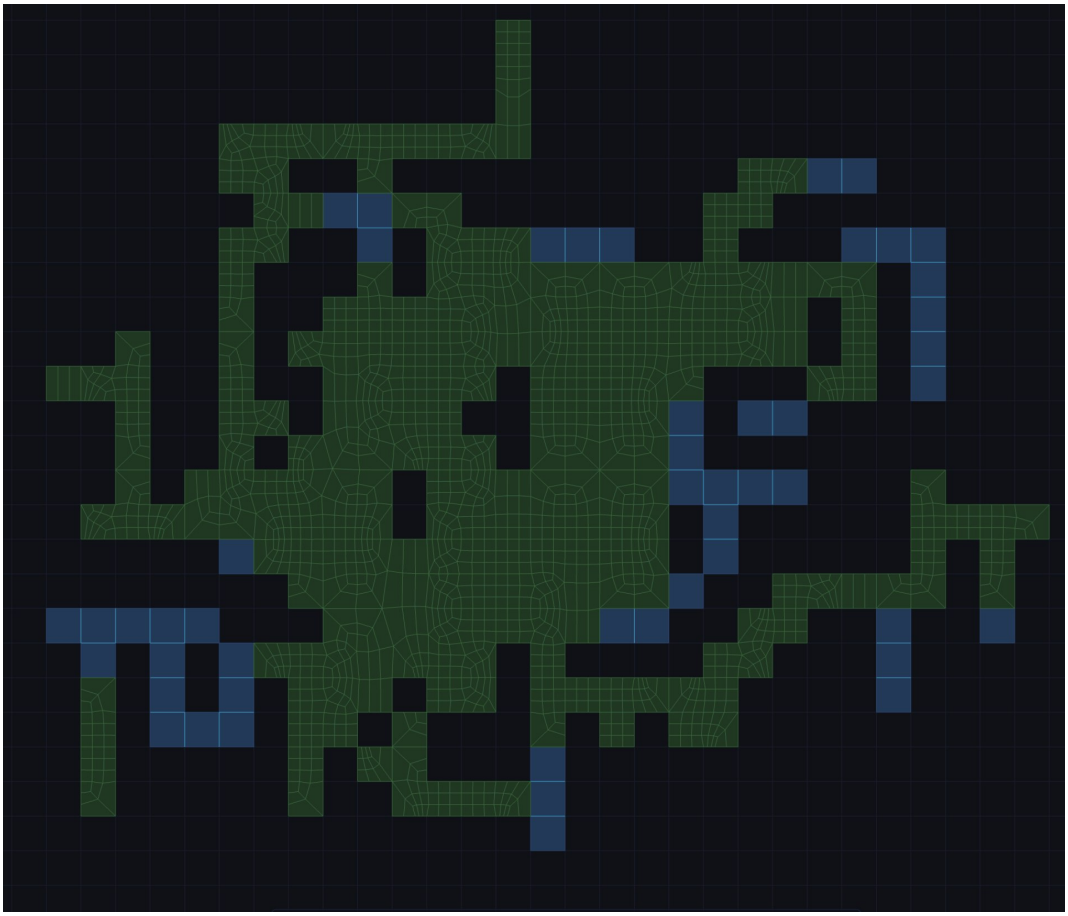


Рис. 36: Нерівномірна 2D-сітка після локального уточнення 3×3 та лапласівського згладжування

### 3.2.2 Метод Taubin

Постановка задачі. Після локального згущення деякі вузли сітки опиняються у «поганих» позиціях: навколишні елементи мають скошені кути і низький *SJ*. Потрібно перемістити вузли так, щоб елементи стали більш правильними, але при цьому не спотворити загальну форму геометрії.

Крок 0: що таке лапласіан вузла. Для кожного внутрішнього вузла $\mathbf{p}_i$ (тривимірна точка в просторі) визначимо його сусідів — усі вузли, з'єднані з $\mathbf{p}_i$ ребром сітки. Нехай таких сусідів $m$ штук, їхні координати $\mathbf{p}_1, \mathbf{p}_2, \ldots, \mathbf{p}_m$.

Лапласіан вузла — це вектор, що вказує від поточної позиції вузла до середнього арифметичного позицій його сусідів:

$$\Delta_i = \underbrace{\frac{1}{m}\sum_{j=1}^{m} \mathbf{p}_j}_{\text{середнє сусідів}} - \mathbf{p}_i \tag{6}$$

Якщо $\Delta_i = \mathbf{0}$ — вузол вже стоїть рівно посередині між сусідами (ідеально). Якщо $\Delta_i \neq \mathbf{0}$ — вузол зміщений відносно оточення, і переміщення у напрямку $\Delta_i$ покращить форму сусідніх елементів.

Чому чистий лапласіан (Лапласівське згладжування) не підходить. Якщо просто переміщувати кожен вузол у напрямку $\Delta_i$ (тобто до середнього сусідів), отримуємо класичне лапласівське згладжування. Але при кожній ітерації крайні вузли (на межі геометрії) теж «притягуються» до центра — сітка повільно стискається всередину. Після десятків ітерацій геометрія об'єкта суттєво спотворюється. Це явище називається усадкою (shrinkage).

Ідея методу Taubin: нейтралізувати усадку зворотним кроком. Метод Taubin виконує дві дії поспіль на кожній ітерації: спочатку «стиснути» (покращити кути елементів), потім одразу «розширити» назад (компенсувати усадку). Якщо крок розширення трохи більший за крок стискання, усадка нейтралізується, а покращення кутів залишається.

Нехай $\lambda > 0$ — величина кроку стискання, $\mu < 0$ — величина кроку розширення (від'ємна, бо напрямок протилежний). Одна ітерація складається з двох кроків.

Крок 1 — стискання (параметр $\lambda = 0.5$). Для кожного внутрішнього вузла обчислюємо лапласіан $\Delta_i$ і переміщуємо вузол на частку $\lambda$ у цьому напрямку:

$$\mathbf{p}_i^{(1)} = \mathbf{p}_i + \lambda \cdot \Delta_i \tag{7}$$

Тут $\mathbf{p}_i^{(1)}$ — нова позиція вузла після першого кроку. Коефіцієнт $\lambda = 0.5$ означає, що вузол переміщується рівно на половину відстані до середнього своїх сусідів. Кути елементів покращуються, але сітка трохи «стиснулась».

Крок 2 — розширення (параметр $\mu = -0.53$). Тепер перераховуємо лапласіан $\Delta_i^{(1)}$ вже з нових позицій $\mathbf{p}^{(1)}$ і знову переміщуємо вузол, але вже у зворотному напрямку

(бо $\mu < 0$):

$$\mathbf{p}_i^{(2)} = \mathbf{p}_i^{(1)} + \mu \cdot \Delta_i^{(1)} \tag{8}$$

Оскільки $|\mu| = 0.53 > \lambda = 0.5$, крок розширення трохи більший за крок стискання. У підсумку за дві дії вузол повертається приблизно на початкове місце за геометрією, але форма елементів покращується. Усадка нейтралізується.

Важливо: схема Якобі. На кожному кроці лапласіан $\Delta_i$ обчислюється для ВСІХ вузлів одночасно, на основі позицій, що були ДО початку кроку. Потім усі вузли переміщуються разом. Це гарантує, що порядок обходу вузлів не впливає на результат.

Переваги методу Taubin: відсутність усадки, значне покращення $SJ$, складність $O(N)$ на пару кроків.

Недолік: не вирівнює розміри елементів — поряд можуть залишатися дрібні та великі гексаедри.

### 3.2.3 Taubin Combined Smoothing

Ідея: вирівнювати не лише кути, а й розміри елементів. Метод Taubin з попереднього розділу усуває усадку і покращує форму кутів, але не вирівнює розміри елементів — після нього поряд можуть залишатися великі і дрібні гексаедри. Taubin Combined Smoothing розширює базовий метод, замінюючи чистий лапласіан на суму двох складових.

Два типи зміщення. Для кожного вузла $\mathbf{p}_i$ вводяться два різних вектори зміщення:

- $\Delta_i^{\text{кут}}$ — лапласіан (той самий, що і в методі Taubin): вектор від $\mathbf{p}_i$ до середнього арифметичного сусідніх вузлів. Переміщення у цьому напрямку вирівнює кути навколишніх гексаедрів.
- $\Delta_i^{\text{розмір}} = \mathbf{c}_i^{\text{зваж}} - \mathbf{p}_i$ — вектор від $\mathbf{p}_i$ до зваженого центроїда суміжних елементів. Переміщення у цьому напрямку зсуває вузол туди, де він краще розподіляє об'єми навколишніх гексаедрів рівномірно.

Зважений центроїд: що це і як обчислюється. Суміжні елементи — усі гексаедри, що містять вузол $\mathbf{p}_i$ серед своїх 8 кутових вузлів. Нехай таких елементів $L$ штук. Для кожного $l$-го суміжного гексаедра позначимо:

- $\mathbf{c}_l$ — геометричний центр $l$-го гексаедра (середнє арифметичне координат його 8 кутових вузлів);
- $V_l$ — об'єм $l$-го гексаедра.

Зважений центроїд:

$$\mathbf{c}_i^{\text{зваж}} = \frac{\sum_{l=1}^{L} V_l \cdot \mathbf{c}_l}{\sum_{l=1}^{L} V_l} \tag{9}$$

Що означає ця формула: більші за об'ємом гексаедри мають більшу «вагу» при обчисленні середнього. Вузол притягується сильніше до центрів великих сусідніх елементів — після переміщення великий елемент «зменшується», а маленький поряд — «збільшується». Це вирівнює розподіл розмірів.

Комбіноване зміщення. Два вектори об'єднуються в один із ваговими коефіцієнтами $\alpha$ і $\beta$:

$$\Delta_i = \alpha \cdot \Delta_i^{\text{кут}} + \beta \cdot \Delta_i^{\text{розмір}}, \quad \alpha = 0.6,\ \beta = 0.4 \tag{10}$$

Чому $\alpha = 0.6$ і $\beta = 0.4$: покращення кутів ($\alpha$) важливіше для точності МСЕ-розрахунку, ніж рівномірність розмірів ($\beta$), тому кутова складова має більшу вагу. Сума $\alpha + \beta = 1$ забезпечує, що вектор $\Delta_i$ залишається адекватно масштабованим.

Далі цей вектор $\Delta_i$ підставляється у ті самі два кроки методу Taubin (стискання з $\lambda = 0.5$, розширення з $\mu = -0.53$) замість чистого лапласіана. Усадка нейтралізується так само, як і у базовому методі.

Переваги: одночасно покращує і кути, і рівномірність розмірів — якість гексаедрів зростає більш комплексно.

Недолік — складність обчислення $V_l$. Для правильного (ненабухлого, неперекрученого) гексаедра об'єм можна обчислити простою формулою. Але після локального згущення виникають перехідні гексаедри з нерівними ребрами та скошеними гранями. Для загального (довільно деформованого) гексаедра точне обчислення об'єму вимагає числового інтегрування:

$$V_l = \int_{\Omega_l} d\Omega \approx \sum_g w_g \cdot \det \mathbf{J}(\xi_g, \eta_g, \zeta_g) \tag{11}$$

де $\mathbf{J}(\xi_g, \eta_g, \zeta_g)$ — матриця Якобіана відображення з канонічного куба $[-1,1]^3$ у реальний тривимірний простір, обчислена у точці квадратури $(\xi_g, \eta_g, \zeta_g)$; $w_g$ — вага цієї точки. Це вимагає підтримки ізопараметричного елемента і числової квадратури — суттєво складніша реалізація порівняно з чистим Taubin, де всі обчислення є лише середніми арифметичними координат.

### 3.2.4 Порівняння та обґрунтування вибору

Табл. 6: Порівняння методів згладжування

| Метод | Усадка | Кути | Розміри | Складність |
|---|---|---|---|---|
| Лапласівський | Є | Добре | Ні | Проста |
| Taubin | Ні | Добре | Ні | Проста |
| Taubin Combined | Ні | Добре | Добре | Складна (обчислення $V_l$) |

Рішення: для реалізації обрано метод Taubin без вирівнювання об'ємів.

Обчислення точних об'ємів скривлених гексаедрів вимагає чисельного інтегрування і суттєво ускладнює реалізацію. Метод Taubin без цього доповнення вже забезпечує значне покращення якості (усунення усадки, покращення $SJ$) і є достатнім для цілей даної роботи. Додавання вирівнювання об'ємів розглядається як напрям подальшого розвитку системи.

### 3.2.5 Деталі реалізації Taubin

Один прохід = пара кроків (стискання + розширення). Виконується 10 пар. Вузли на зовнішній межі геометрії не переміщуються.

Після кожного кроку перевіряється Quality Guard: якщо $SJ$ будь-якого суміжного елемента падає нижче $0.2$ (і до кроку він був $\geq 0.2$) — зміщення відхиляється:

$$SJ(H_l) \geq 0.2 \quad \text{для всіх суміжних } H_l \tag{12}$$

Очікуваний результат після 10 пар: $SJ_{\min} \geq 0.5$ у перехідній зоні (проти $SJ \geq 0.3$ одразу після згущення).

## 3.3 Метрики якості та рекомендовані значення

Табл. 7: Метрики якості гексаедральних елементів

| Метрика | Ідеал | Допустимо | Неприпустимо |
|---|---|---|---|
| Масштабований якобіан ($SJ$) | 1.0 | $\geq 0.2$ | $\leq 0$ |
| Співвідношення сторін ($AR$) | 1.0 | $\leq 5.0$ | $> 10$ |
| Мінімальний кут (2D) | $90°$ | $\geq 45°$ | $< 20°$ |
| Максимальний кут (2D) | $90°$ | $\leq 135°$ | $> 160°$ |

## 3.6 Зв'язок якості сітки з точністю МСЕ

Покращення геометричної якості елементів після уточнення безпосередньо впливає на точність МСЕ-розрахунків. Деградація якості проявляється у двох аспектах.

По-перше, числова нестійкість: при $SJ \to 0$ якобіан $\det(\mathbf{J}) \to 0$, що спричиняє погану обумовленість матриці $K^{(e)}$. Числові похибки при розв'язанні системи зростають.

По-друге, похибка апроксимації: для сильно деформованих елементів функції форми погано апроксимують поле переміщень, що збільшує дискретизаційну похибку навіть при дрібній сітці.

Утримання $SJ \geq 0.2$ через захисну перевірку при згладжуванні гарантує прийнятний рівень числової точності у всіх елементах.

## 3.7 Висновки до розділу

У розділі описано методи покращення якості сітки після уточнення.

1. Проведено порівняльний аналіз методів згладжування: лапласівський, Taubin та Taubin Combined. Показано, що лапласівський метод має проблему усадки, а Taubin Combined потребує складного обчислення об'ємів скривлених гексаедрів.

2. Для майбутньої реалізації обрано метод Taubin ($\lambda = 0.5$, $\mu = -0.53$, 10 пар ітерацій) із захистом за масштабованим якобіаном (поріг $SJ_{\min} = 0.2$).

3. Граничні вузли залишаються незмінними для збереження геометрії домену.

4. Захист від інверсії та деградації забезпечує придатність сітки для МСЕ-розрахунків після будь-якої конфігурації уточнення.

# ВИСНОВКИ

У курсовій роботі розроблено та програмно реалізовано систему локального адаптивного уточнення гексаедральних сіток. За результатами роботи отримано такі основні результати:

1. Проведено критичний аналіз восьми провідних систем генерації та уточнення скінченно-елементних сіток. Встановлено, що жодна з них не реалізує шаблонного локального уточнення вже побудованої гексаедральної сітки без повної перегенерації. Відкриті системи обмежені тетраедральними сітками, а комерційні є недоступними через ціну і відсутність відкритого API. Це підтвердило актуальність та необхідність власної розробки.

2. Розроблено ефективну топологічну структуру даних для гексаедральних сіток, що підтримує швидкий обхід сусідів через шість посилань (Neighbor-Left/Right/Bottom/Top/Front/Back), ланцюжкову нумерацію елементів (Build-Chains) та ієрархію батьківських і дочірніх розбиттів без прив'язки до координатної сітки.

3. Запропоновано та реалізовано алгоритм локального адаптивного уточнення на основі шаблонного підходу для двовимірних і тривимірних сіток. Алгоритм підтримує три типи шаблонів (T1, T2, T4), реалізує трипрохідну схему (I-, J-, K-прохід) та гарантує конформність результуючої сітки без повторної генерації.

4. Розроблено метод поширення вузлів (Node Propagation) як механізм усунення каскадного застосування шаблонів. Без цього механізму ізольований T1 на крайовому вузлі породжує дочірні елементи з гострими кутами (Scaled Jacobian нижче 0.3). Node Propagation перетворює T1 $\to$ T2 (SJ $\geq 0.5$) або T2 $\to$ T4 (SJ $\approx$ SJ батька), зберігаючи геометричну якість.

5. Досліджено методи покращення геометричної якості сітки після уточнення. Лапласівське згладжування із захистом від інверсії вузлів реалізовано для двовимірного випадку. Для тривимірних сіток проведено порівняльний аналіз методів (Laplacian, Taubin, Taubin Combined) та обґрунтовано вибір методу Taubin ($\lambda = 0.5$, $\mu = -0.53$) як оптимального для подальшої реалізації.

6. Розроблено веб-систему CadEditor на базі .NET 8 / Angular 17 / Three.js, що надає інтерактивний браузерний інтерфейс для побудови, локального уточнення та редагування гексаедральних сіток. Система підтримує транзакційний механізм сесії (begin/cancel/commit), експорт у форматах Gmsh (.msh), VTK (.vtk) та Abaqus (.inp) і має відкрите REST API.

Перспективи подальшого розвитку. Система CadEditor перебуває в активній розробці: значна частина описаних нижче напрямів вже реалізована у вигляді окремих прототипів або модулів, проте ще не інтегрована в єдину систему і не готова до публічного представлення. Основні пріоритети:

1. Інтеграція МСЕ-модуля (найвищий пріоритет). Паралельно з CadEditor розроблено окремий МСЕ-розрахунковий модуль на базі 20-вузлових серендипових гексаедрів, який вже вміє будувати сітку, виконувати розрахунок напружень і відображати теплову карту Мізеса у браузері (готовність — близько 80–90%). Головний наступний крок — передача сітки між двома системами і об'єднання в єдиний конвеєр «побудова сітки → розрахунок → візуалізація».

2. Тривимірне згладжування. Реалізація методу Taubin для 3D-сіток із захистом за масштабованим якобіаном завершить повний конвеєр уточнення+якість. Алгоритм вибрано та обґрунтовано в розділі 3; залишається програмна реалізація.

3. Фізичний простір та криволінійні елементи. Ізопараметричні перетворення для криволінійних гексаедрів дозволять точно апроксимувати складні геометричні форми й розширять область застосування системи за межі прямолінійних сіток.

4. Імпорт геометричних оболонок та автозаповнення. Розробляється підтримка імпорту поверхневих моделей (Blender, Fusion 360 та ін.) з автоматичним заповненням об'єму гексаедральними супер-елементами. Поточна реалізація готова орієнтовно на 30%. Кінцева мета — користувач задає лише бажану густину елементів, система будує сітку самостійно.

5. Автоматизація конвеєру та AI-асистент. Автоматичне виявлення критичних зон для уточнення, AI-рекомендації щодо параметрів розбиття ($n_I$, $n_J$, $n_K$) і порогів якості, а також природномовний інтерфейс для опису геометрії замість ручного розміщення елементів.

6. Платформа та розширення форматів. Повноцінний веб-сайт з авторизацією і хмарним збереженням проектів, розширення підтримки форматів імпорту/експорту для сумісності з основними CAD/FEM системами ринку. Базовий імпорт/експорт вже частково реалізовано.

# СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ


[1] Geuzaine C., Remacle J.-F. Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. International Journal for Numerical Methods in Engineering. 2009. Vol. 79(11). P. 1309–1331.

[2] ANSYS Inc. ANSYS Meshing User's Guide. Release 2024 R1. Canonsburg, PA: ANSYS Inc., 2024. 416 p.

[3] Dassault Systèmes Simulia Corp. Abaqus Analysis User's Manual. Version 2024. Providence, RI: Dassault Systèmes, 2024.

[4] COMSOL AB. COMSOL Multiphysics Reference Manual. Version 6.2. Stockholm: COMSOL AB, 2023. 1724 p.

[5] Coreform LLC. Coreform Cubit User Documentation. Version 2024.3. Salt Lake City: Coreform, 2024.

[6] Blacker T. D., Stephenson M. B. Paving: A new approach to automated quadrilateral mesh generation. International Journal for Numerical Methods in Engineering. 1991. Vol. 32(4). P. 811–847.

[7] Open CASCADE Technology. OCCT Reference Manual. Version 7.8. URL: `https://dev.opencascade.org/doc/refman/html/`.

[8] Weller H. G., Tabor G., Jasak H., Fureby C. A tensorial approach to computational continuum mechanics using object-oriented techniques. Computers in Physics. 1998. Vol. 12(6). P. 620–631.

[9] FreeCAD Community. FreeCAD FEM Workbench Documentation. URL: `https://wiki.freecad.org/FEM_Workbench`. 2024.

[10] Taubin G. A signal processing approach to fair surface design. Proceedings of SIGGRAPH '95. New York: ACM, 1995. P. 351–358.

[11] Schneiders R. A grid-based algorithm for the generation of hexahedral element meshes. Engineering with Computers. 1996. Vol. 12(3). P. 168–177.

[12] Owen S. J. A Survey of Unstructured Mesh Generation Technology. Proceedings of the 7th International Meshing Roundtable. Dearborn, MI: Sandia National Laboratories, 1998. P. 239–267.

[13] Benzley S. E., Perry E., Merkley K., Clark B., Sjaardema G. A Comparison of All-Hexahedral and All-Tetrahedral Finite Element Meshes for Elastic and Elasto-plastic Analysis. Proceedings of the 4th International Meshing Roundtable. Sandia National Laboratories, 1995. P. 179–191.

[14] Knupp P. M. Algebraic mesh quality metrics. SIAM Journal on Scientific Computing. 2001. Vol. 23(1). P. 193–218.

[15] Zienkiewicz O. C., Taylor R. L., Zhu J. Z. The Finite Element Method: Its Basis and Fundamentals. 7th ed. Oxford: Butterworth-Heinemann, 2013. 756 p.

[16] Mitchell S. A., Tautges T. J. Pillowing doublets: Refining a mesh to ensure that faces share at most one edge. Proceedings of the 4th International Meshing Roundtable. Sandia National Laboratories, 1995. P. 231–240.

[17] Canann S. A., Tristano J. R., Staten M. L. An Approach to Combined Laplacian and Optimization-Based Smoothing for Triangular, Quadrilateral, and Quad-Dominant Meshes. Proceedings of the 7th International Meshing Roundtable. Dearborn, MI: Sandia National Laboratories, 1998. P. 479–494.

[18] Freitag L. A., Plassmann P. Local optimization-based simplicial mesh untangling and improvement. International Journal for Numerical Methods in Engineering. 2000. Vol. 49(1–2). P. 109–125.

[19] Parrish M., Borden M. J., Staten M. L., Benzley S. E. A selective approach to conformal refinement of unstructured hexahedral finite element meshes. Proceedings of the 16th International Meshing Roundtable. Seattle, WA: Springer, 2007. P. 251–268.

[20] Shephard M. S., Georges M. K. Automatic three-dimensional mesh generation by the finite octree technique. International Journal for Numerical Methods in Engineering. 1991. Vol. 32(4). P. 709–749.

[21] Ito Y., Shih A. M., Soni B. K. Octree-based reasonable-quality hexahedral mesh generation using a new set of refinement templates. International Journal for Numerical Methods in Engineering. 2009. Vol. 77(13). P. 1809–1833.

[22] Microsoft Corporation. ASP.NET Core documentation. URL: `https://learn.microsoft.com/aspnet/core`. 2024.

[23] Three.js contributors. Three.js documentation. URL: `https://threejs.org/docs/`. 2024.

[24] Vollmer J., Mencl R., Müller H. Improved Laplacian Smoothing of Noisy Surface Meshes. Computer Graphics Forum. 1999. Vol. 18(3). P. 131–138.

[25] Angular Team. Angular documentation. URL: `https://angular.dev/`. 2024.